

Dynamic Programming Templates

Six template families. Every problem maps to one loop skeleton and one dp-state definition.

Quick Reference Table

#	Name	Description	Intuition	Time	Space	Key Implementation
70	Climbing Stairs	Each step you can climb 1 or 2 steps. How many distinct ways to reach step n ?	the last move was a 1-step or a 2-step, so ways to reach n = ways to $n-1$ + ways to $n-2$.	$O(n)$	$O(n)$	Standard
198	House Robber	Rob houses on a line — can't rob two adjacent. Maximize total.	at each house, either skip it (keep $dp[i-1]$) or rob it ($dp[i-2] + value$, since $i-1$ is now off-limits).	$O(n)$	$O(n)$	Standard
213	House Robber II	Houses arranged in a circle — first and last are adjacent. Can't rob both.	breaking the circle means either the first house is excluded or the last is — solve both lines and keep the better.	$O(n)$	$O(1)$	Standard
300	Longest Increasing Subsequence	Length of the longest strictly increasing subsequence.	the best subsequence ending at i extends the longest earlier one whose last value is still smaller.	$O(n^2)$	$O(n)$	Standard
1339	Word Break	Can s be segmented into a sequence of dictionary words?	a prefix is breakable if some earlier breakable cut leaves a dictionary word as the final piece.	$O(n^2 \cdot L)$ where L = substring length	$O(n)$	Standard
416	Partition Equal Subset Sum	Can the array be partitioned into two subsets with equal sum?		$O(n)$	$O(sum)$	Standard
494	Target Sum	Assign $+$ or $-$ to each number to reach <code>target</code> . How many ways?		$O(n)$	$O(sum)$	Standard
474	Ones and Zeroes	Given strings of '0'/'1', find the largest subset using at most m zeros and n ones.		$O(l)$	$O(mn)$	Standard
322	Coin Change	Minimum number of coins to make <code>amount</code> . Coins can be reused.		$O(amount)$	$O(amount)$	Standard
518	Coin Change II	Number of distinct combinations (order doesn't matter) to make <code>amount</code> .		$O(amount)$	$O(amount)$	Standard
377	Combination Sum IV	Number of ordered sequences (order matters) that sum to <code>target</code> .		$O(target)$	$O(target)$	Standard
1143	Longest Common Subsequence	Length of the longest subsequence present in both strings.	if the last chars match, they're part of the LCS (+1 on the diagonal); otherwise drop one char from whichever string and keep the better.	$O(m \cdot n)$	$O(m \cdot n)$	Standard
72	Edit Distance	Minimum operations (insert, delete, replace) to convert <code>word1</code> to <code>word2</code> .	matched chars cost nothing (diagonal); otherwise take the cheapest of replace/delete/insert and add 1.	$O(m \cdot n)$	$O(m \cdot n)$	Standard
115	Distinct Subsequences	Number of ways s contains t as a subsequence.	you can always skip the current char of s ; if it matches the current char of t , you may also use it to advance t .	$O(m \cdot n)$	$O(m \cdot n)$	Standard
647	Palindromic Substrings	Count all palindromic substrings of s .	$s[i..j]$ is a palindrome when its two ends match AND the inside $s[i+1..j-1]$ already is.	$O(n^2)$	$O(n^2)$	Standard
516	Longest Palindromic Subsequence	Length of the longest subsequence of s that is a palindrome.	matching ends add 2 to the inner range's best; otherwise drop one end and keep the larger.	$O(n^2)$	$O(n^2)$	Standard
312	Burst Balloons	Burst all balloons to maximize coins. Coins from bursting balloon k between open boundaries i and j = $balloons[i] * balloons[k] * balloons[j]$.		$O(n^3)$	$O(n^2)$	Standard
62	Unique Paths	Number of paths from top-left to bottom-right, moving only right or down.	you reach a cell only from above or from the left, so its path count is the sum of those two.	$O(m \cdot n)$	$O(m \cdot n)$	Standard
64	Minimum Path Sum	Minimum sum path from top-left to bottom-right, moving only right or down.	the cheapest way into a cell is its own value plus the cheaper of the cell above or to its left.	$O(m \cdot n)$	$O(m \cdot n)$	Standard
221	Maximal Square	Find the largest all-1 square in a binary matrix. Return its area.		$O(m \cdot n)$	$O(m \cdot n)$	Standard

#	Name	Description	Intuition	Time	Space	Key Implementation
1						
3 2 9	Longest Increasing Path in a Matrix	Find the longest strictly increasing path. Can move in all 4 directions.	strictly increasing values forbid revisiting, so each cell's longest path is 1 + the best of its larger neighbors — cache it.	$O(m \cdot n)$	$O(m \cdot n)$	Standard
1 2 1	Best Time to Buy and Sell Stock (at most 1 transaction)		hold is the best profit if you currently own a share bought at the lowest price seen; sell whenever today's price beats that.	$O(n)$	$O(1)$	Standard
1 2 2	Best Time to Buy and Sell Stock II (unlimited transactions)		with unlimited trades, buying can fund itself from accumulated cash, so capture every upward move.	$O(n)$	$O(1)$	Standard
1 2 3	Best Time to Buy and Sell Stock III (at most 2 transactions)		the second buy can only spend profit left after the first sell, so chain the states in trade order.	$O(n)$	$O(1)$	Standard
3 0 9	Best Time to Buy and Sell Stock with Cooldown	After selling, must wait 1 day before buying again.	the cooldown forces buying to draw from cash that's at least one day old, so carry yesterday's cash forward.	$O(n)$	$O(1)$	Standard
7 1 4	Best Time to Buy and Sell Stock with Transaction Fee	Pay a fee per transaction (on sell). Unlimited transactions.	the fee just shrinks every sale, so a trade is only worth making when the gain clears the fee.	$O(n)$	$O(1)$	Standard
7 4 6	Min Cost Climbing Stairs	Each step has a cost. You can start from index 0 or 1, and from each step you can climb 1 or 2 steps. Return the minimum cost to reach the top (past the last index).		$O(n)$	$O(n)$	Standard
9 1	Decode Ways	A string of digits can be decoded: '1'-'9' → single letter, "10"-"26" → two-digit letter. Count total decoding ways.		$O(n)$	$O(n)$	Two sources at each position. If the current digit is non-zero, it can be decoded alone ($dp[i] += dp[i-1]$). If the previous two digits form a valid two-letter code (10-26), add $dp[i-2]$.
2 7 9	Perfect Squares	Return the minimum number of perfect squares (1, 4, 9, 16, ...) that sum to n .		$O(n\sqrt{n})$	$O(n)$	iterate all squares $\leq i$
9 6	Unique Binary Search Trees	Return the number of structurally unique BSTs that store values 1 to n.		$O(n^2)$	$O(n)$	Catalan number recurrence. $dp[i]$ = number of unique BSTs with i nodes. When root = j, left subtree has j-1 nodes, right subtree has i-j nodes: $dp[i] += dp[j-1] * dp[i-j]$.
6 3	Unique Paths II	A robot moves from top-left to bottom-right in an m×n grid with obstacles. Count unique paths. Cells with <code>obstacleGrid[i][j] == 1</code> are blocked.		$O(m \cdot n)$	$O(m \cdot n)$	Same as #62 (Unique Paths) but skip cells with obstacles: $dp[i][j] = 0$ if blocked, else $dp[i-1][j] + dp[i][j-1]$.
5 8 3	Delete Operation for Two Strings	Return the minimum number of deletions to make word1 and word2 equal.		$O(m \cdot n)$	$O(m \cdot n)$	Reduce to LCS. Minimum deletions = $len(word1) + len(word2) - 2 * LCS(word1, word2)$. Compute LCS using standard 2D DP.
3 6 8	Largest Divisible Subset	Find the largest subset of distinct positive integers such that for every pair (a, b), either $a \% b == 0$ or $b \% a == 0$.		$O(n^2)$	$O(n)$	Sort first, then apply LIS-style DP. $dp[i]$ = size of largest divisible subset ending at <code>nums[i]</code> . If $nums[i] \% nums[j] == 0$, then $dp[i] = \max(dp[i], dp[j] + 1)$. Track parent indices to reconstruct the subset.
9 3 1	Minimum Falling Path Sum	Given an n×n matrix, return the minimum sum of a falling path. A falling path starts at any element in the first row and chooses the element directly below or diagonally adjacent in each row.		$O(n^2)$	$O(n^2)$	Grid DP with three sources from the row above. $dp[i][j] = matrix[i][j] + \min(dp[i-1][j-1], dp[i-1][j], dp[i-1][j+1])$.
1 0	Regular Expression Matching	Implement regex matching with <code>.</code> (any single char) and <code>*</code> (zero or more of the preceding element). Match must cover the entire input string.		$O(m \cdot n)$	$O(m \cdot n)$	2D DP. $dp[i][j]$ = whether <code>s[0..i]</code> matches <code>p[0..j]</code> . The <code>*</code> has two cases: zero occurrences ($dp[i][j-2]$) or one-more occurrence when the preceding pattern char matches <code>s[i-1]</code> .

#	Name	Description	Intuition	Time	Space	Key Implementation
8 7	Scramble String	Given s1 and s2, return true if s2 is a scrambled string of s1 (formed by recursively partitioning into two non-empty substrings and optionally swapping them).		$O(n^4)$	$O(n^3)$	memoized recursion over (i1, i2, len). At each length, try every split point both swapped and non-swapped.
1 0 4 9	Last Stone Weight II	Repeatedly smash the two heaviest stones (result is their difference). Return the smallest possible weight of the remaining stone.		$O(n \cdot \text{sum})$	$O(\text{sum})$	equivalent to splitting stones into two groups to minimize the difference of their sums. 0/1 knapsack to find the max subset sum $\leq \text{total}/2$.
1 3 1 2	Minimum Insertion Steps to Make a String Palindrome	Return the minimum number of insertions needed to make a string a palindrome.		$O(n^2)$	$O(n^2)$	answer = $n - \text{longest palindromic subsequence (LPS)}$. LPS = LCS of the string and its reverse; or solve directly via interval DP.
4 4	Wildcard Matching	Match string s against pattern p with ? (any single char) and * (any sequence including empty).	* either consumes one more char of s (dp[i-1][j]) or matches empty (dp[i][j-1]); everything else is a 1-to-1 char/ ? match.	$O(m \cdot n)$	$O(m \cdot n)$	"" matches empty prefix
5 3	Maximum Subarray	Find the contiguous subarray with the largest sum.	the best sum ending at i either extends the previous best or restarts at nums[i] (Kadane).	$O(n)$	$O(1)$	Kadane restart vs extend
9 7	Interleaving String	Return whether s3 is formed by interleaving s1 and s2 while preserving each one's order.	the last char of s3[0..i+j) comes from either s1 or s2 ; reuse the answer for the smaller prefix.	$O(m \cdot n)$	$O(m \cdot n)$	take from s1
1 2 0	Triangle	Find the minimum path sum from top to bottom, moving to adjacent indices on the row below.	working bottom-up, each cell's best is its value plus the cheaper of the two children directly below.	$O(n^2)$	$O(n)$	bottom-up over rows
1 3 2	Palindrome Partitioning II	Return the minimum number of cuts so that every substring of the partition is a palindrome.	if s[j..i] is a palindrome, a cut after j-1 lets cuts[i] = cuts[j-1] + 1 ; precompute palindromes with interval DP.	$O(n^2)$	$O(n^2)$	interval DP precompute palindromes
1 4 0	Word Break II	Return all sentences obtained by segmenting s into space-separated dictionary words.	memoize on each suffix the list of valid segmentations, prepending each matching word to the segmentations of the remainder.	$O(n^2 \cdot 2^n)$ worst case	$O(2^n)$	memoized DFS returns sentences
1 5 2	Maximum Product Subarray	Find the contiguous subarray with the largest product.	a negative number flips max and min, so track both; the running max product ending at i may come from the prior max or min.	$O(n)$	$O(1)$	negative swaps max/min
1 7 4	Dungeon Game	Find the minimum starting health so the knight survives from top-left to bottom-right (each cell adds/subtracts health, must stay ≥ 1).	health needed depends on the future, so fill from the bottom-right: needed entering a cell = max(1, min child need - cell value).	$O(m \cdot n)$	$O(m \cdot n)$	fill from bottom-right
1 8 8	Best Time to Buy and Sell Stock IV	Maximize profit with at most k transactions.	for each allowed transaction count t , track best buy and sell; this is #123 generalized to k chained state pairs.	$O(n \cdot k)$	$O(k)$	k chained state pairs
2 6 4	Ugly Number II	Find the nth ugly number (positive integers whose only prime factors are 2, 3, 5).	every ugly number is a previous ugly number times 2, 3, or 5; merge the three multiples with pointers.	$O(n)$	$O(n)$	merge multiples via pointers
3 1 3	Super Ugly Number	Find the nth super ugly number whose prime factors all come from a given primes list.	generalize #264 to arbitrary primes — keep one pointer per prime and pick the minimum next candidate.	$O(n \cdot k)$	$O(n + k)$	one pointer per prime
3 3 7	House Robber III	Houses form a binary tree; you cannot rob two directly-linked houses. Maximize the total.	each node returns two values — best if it is robbed (skip children) vs not robbed (children free to choose).	$O(n)$	$O(h)$	tree DP returns {notRob, rob}
3 4 3	Integer Break	Break n into the sum of at least two positive integers maximizing their product.	for each split j , the rest can stay as i-j or be broken further (dp[i-j]); take the best.	$O(n^2)$	$O(n)$	split point, break further or not
3 7 5	Guess Number Higher or Lower II	Find the minimum amount of money to guarantee a win when guessing a number in [1, n] , paying the guessed value on each wrong guess.	for range [i, j] , guessing k costs k plus the worst of the two resulting subranges; minimize over k .	$O(n^3)$	$O(n^2)$	interval DP over range length
3 7 6	Wiggle Subsequence	Find the length of the longest subsequence whose consecutive differences strictly alternate between positive and negative.	track the best subsequence ending with an up-move and with a down-move; each rise extends down , each fall extends up .	$O(n)$	$O(1)$	rise extends down-ending seq

#	Name	Description	Intuition	Time	Space	Key Implementation
4 1 3	Arithmetic Slices	Count the number of arithmetic subarrays (length ≥ 3 , constant difference).	if <code>nums[i]</code> continues the arithmetic run ending at <code>i-1</code> , it adds <code>dp[i-1]+1</code> new slices ending at <code>i</code> .	$O(n)$	$O(1)$	extend arithmetic run
4 4 6	Arithmetic Slices II - Subsequence	Count the arithmetic subsequences (length ≥ 3) of the array.	for each pair (j, i) with difference <code>d</code> , weak slices ending at <code>i</code> extend those ending at <code>j</code> ; promote weak to valid when length reaches 3.	$O(n^2)$	$O(n^2)$	per-index map keyed by difference
4 8 6	Predict the Winner	Two players alternately take a number from either end; decide whether player 1 can win (score $\geq$ player 2).	on range $[i, j]$ the current player maximizes value – opponent's best on the remaining range.	$O(n^2)$	$O(n^2)$	interval DP, score difference
5 0 9	Fibonacci Number	Return the n th Fibonacci number.	each term is the sum of the previous two — the canonical linear 1D recurrence.	$O(n)$	$O(1)$	Standard
5 5 2	Student Attendance Record II	Count length- n attendance records that are rewardable (at most one 'A', no three consecutive 'L'), modulo $1e9+7$.	state = (number of 'A's so far 0/1, trailing 'L' run 0/1/2); each day append P, A, or L respecting limits.	$O(n)$	$O(1)$	state = (A count, trailing L run)
5 7 6	Out of Boundary Paths	Count paths that move a ball off the grid in exactly <code>maxMove</code> steps from a start cell, modulo $1e9+7$.	moving off the boundary counts as one path; otherwise spread current-step counts to 4 neighbors for the next step.	$O(\max M \text{ ove-} m \cdot n)$	$O(m \cdot n)$	spread to 4 neighbors
6 0 0	Non-negative Integers without Consecutive Ones	Count integers in $[0, n]$ whose binary representation has no two adjacent 1 bits.	scan bits high-to-low; precompute Fibonacci-style counts of valid suffixes, adding them when a free bit is 1, and stop early if two consecutive 1s appear in <code>n</code> .	$O(1)$ (32 bits)	$O(1)$	<code>fib[i]</code> = valid i -bit strings
6 2 9	K Inverse Pairs Array	Count permutations of $1..n$ with exactly <code>k</code> inverse pairs, modulo $1e9+7$.	inserting <code>n</code> into a permutation of $1..n-1$ adds $0..n-1$ inversions; this gives a sliding-window sum over the previous row.	$O(n \cdot k)$	$O(n \cdot k)$	prefix sums of previous row
6 3 8	Shopping Offers	Buy exactly <code>needs[i]</code> of each item at minimum cost, given unit prices and special bundle offers (each usable multiple times).	treat the remaining needs vector as the state; either buy everything at unit price, or apply an affordable offer and recurse.	$O(\text{offers} \cdot \prod \text{needs}_i)$	$O(\prod \text{needs}_i)$	DFS+memo over needs vector
6 3 9	Decode Ways II	Count decodings of a digit string where <code>*</code> is a wildcard for digits 1-9, modulo $1e9+7$.	extend #91 — each position can decode one char (count 1-9 options) or two chars (count valid 10-26 combinations); <code>*</code> multiplies the options.	$O(n)$	$O(n)$	single char, count options
6 5 0	2 Keys Keyboard	Starting with one 'A', using only Copy-All and Paste, return the minimum operations to obtain <code>n</code> 'A's.	the answer is the sum of <code>n</code> 's prime factors — building <code>i</code> from a divisor <code>j</code> costs <code>i/j</code> operations.	$O(n \sqrt{n})$	$O(n)$	build i from divisor j
6 7 3	Number of Longest Increasing Subsequence	Count the number of longest strictly increasing subsequences.	alongside LIS length, track how many ways achieve it; a longer extension resets the count, an equal-length extension accumulates it.	$O(n^2)$	$O(n)$	longer extension resets count
6 7 4	Longest Continuous Increasing Subsequence	Find the length of the longest continuous (contiguous) strictly increasing subarray.	extend the current run while values rise, otherwise restart; a simpler linear scan of #300.	$O(n)$	$O(1)$	contiguous run, reset on drop
6 8 8	Knight Probability in Chessboard	Return the probability that a knight remains on an $n \times n$ board after exactly <code>k</code> moves from a start cell, each move chosen uniformly among 8.	spread the probability at each cell to its 8 knight-move targets per step; off-board moves lose probability.	$O(k \cdot n^2)$	$O(n^2)$	8 knight moves, each prob 1/8
7 1 8	Maximum Length of Repeated Subarray	Find the length of the longest subarray common to two arrays (contiguous).	unlike LCS, the match must be contiguous, so a mismatch resets the running length to 0.	$O(m \cdot n)$	$O(m \cdot n)$	contiguous, no carry on mismatch
7 4 0	Delete and Earn	Deleting <code>nums[i]</code> earns its value but removes all copies of <code>nums[i]-1</code> and <code>nums[i]+1</code> ; maximize earnings.	bucket points by value, then it becomes House Robber over the value line — you cannot take adjacent values.	$O(n + \max)$	$O(\max)$	bucket points by value
8 7 3	Length of Longest Fibonacci Subsequence	Given a strictly increasing array, find the length of the longest Fibonacci-like subsequence (each term = sum of the previous two).	a fib-like subseq is determined by its last two elements (j, i) ; look up whether the required predecessor <code>arr[i]-arr[j]</code> exists earlier.	$O(n^2)$	$O(n^2)$	predecessor before <code>arr[j]</code>

#	Name	Description	Intuition	Time	Space	Key Implementation
887	Super Egg Drop	With k eggs and n floors, return the minimum number of moves to determine the critical floor in the worst case.	invert the problem — $dp[m][k] =$ highest floor count solvable with m moves and k eggs; a move either breaks (test below) or survives (test above), plus the current floor.	$O(k \log n)$	$O(n \cdot k)$	$dp[m][k] =$ floors solvable in m moves
918	Maximum Sum Circular Subarray	Find the maximum subarray sum where the array is circular (subarray may wrap around).	the answer is either a normal max subarray (Kadane) or the total minus the minimum subarray (wrap); guard the all-negative case.	$O(n)$	$O(1)$	track min subarray for wrap
935	Knight Dialer	Count distinct phone numbers of length n dialable by knight moves on a phone keypad, modulo $1e9+7$.	from each digit a knight can reach a fixed set of digits; spread counts across the adjacency map each step.	$O(n)$	$O(1)$	knight adjacency per digit
983	Minimum Cost For Tickets	Given travel days and the cost of 1-day, 7-day, and 30-day passes, return the minimum cost to cover all travel days.	on a travel day choose the cheapest pass covering it by looking back 1, 7, or 30 days; non-travel days inherit the prior cost.	$O(\text{lastDay})$	$O(\text{lastDay})$	non-travel day inherits cost
1027	Longest Arithmetic Subsequence	Find the length of the longest arithmetic subsequence (constant difference).	for each pair (j, i) , the arithmetic subseq ending at i with difference d extends the one ending at j .	$O(n^2)$	$O(n^2)$	per-index map keyed by difference
1048	Longest String Chain	Find the longest chain of words where each word is a predecessor of the next (insert exactly one char).	sort by length; for each word, try removing each character to find a shorter predecessor and extend its best chain.	$O(n \cdot L^2)$	$O(n)$	process shortest first
1137	N-th Tribonacci Number	Return the n th Tribonacci number (each term is the sum of the previous three).	the linear 1D recurrence with three rolling variables instead of two.	$O(n)$	$O(1)$	sum of previous three
1156	Valid Palindrome III	Return whether s becomes a palindrome after removing at most k characters.	the minimum removals to make s a palindrome equals $n - LPS(s)$; check whether that is $\leq k$.	$O(n^2)$	$O(n^2)$	interval DP for LPS
1181	Longest Arithmetic Subsequence of Given Difference	Find the longest arithmetic subsequence with a fixed difference <code>difference</code> .	with a known difference, the predecessor of <code>num</code> must be <code>num - difference</code> ; one hashmap pass suffices.	$O(n)$	$O(n)$	fixed difference, value-keyed map
1246	Number of Ways to Stay in the Same Place After Some Steps	Count the ways to return to index 0 after <code>steps</code> moves (stay, left, or right) without leaving an array of size <code>arrLen</code> , modulo $1e9+7$.	position can never exceed <code>steps/2</code> , so cap the reachable range; each step a position spreads to stay/left/right.	$O(\text{steps} \cdot \min(\text{arrLen}, \text{steps}))$	$O(\min(\text{arrLen}, \text{steps}))$	cap reachable positions
1325	Count Number of Teams	Count triplets (i, j, k) with $i < j < k$ whose ratings are strictly increasing or strictly decreasing.	fix the middle soldier j ; the answer is its number of smaller-before $\times$ larger-after, plus the mirror case.	$O(n^2)$	$O(1)$	fix middle soldier
1447	Maximum Length of Subarray With Positive Product	Find the length of the longest subarray whose product is positive.	track the lengths of positive-product and negative-product runs ending at i ; a negative number swaps them, a zero resets both.	$O(n)$	$O(1)$	zero resets both runs
1696	Jump Game VI	From index 0, each move jumps 1 to k indices forward; maximize the sum of values landed on, reaching the last index.	$dp[i] =$ best score reaching $i = \text{nums}[i] + \max dp[j]$ in the window $[i-k, i-1]$; a monotonic queue keeps that window max in $O(1)$.	$O(n)$	$O(n)$	monotonic queue of indices
1704	Egg Drop With 2 Eggs and N Floors	With exactly 2 eggs and n floors, return the minimum number of moves to determine the critical floor in the worst case.	$dp[i] =$ min worst-case moves for i floors; dropping at floor j costs $1 + \max(dp[j-1] \text{ from break, } dp[i-j] \text{ from survive})$.	$O(n^2)$	$O(n)$	first drop at floor j

All Six Templates

```
// 1. LINEAR 1D – each cell depends on a fixed number of previous cells
// MENTAL MODEL: the answer at i is a fixed recipe over the last one or two answers.
// WHEN: "ways/cost to reach step i", "can't pick adjacent"
int[] dp = new int[n + 1];
dp[0] = base;
for (int i = 1; i <= n; i++)
    dp[i] = f(dp[i-1], dp[i-2], ...);

// 2A. LOOK-BACK 1D – dp[i] depends on all j < i
// MENTAL MODEL: to finish position i, scan every earlier position j and extend the best valid one.
// WHEN: "longest increasing/chain ending here", "can the prefix be segmented"
int[] dp = new int[n];
for (int i = 0; i < n; i++) {
    for (int j = 0; j < i; j++) {
        dp[i] = f(dp[i], dp[j]); // e.g., LIS, word break
    }
}

// -----
// MENTAL MODEL: both are the same 2D recurrence collapsed to 1D.
// dp[i][j] = ways to make sum j using the first i items.
// After dropping the i dimension, dp[j-nums[i]] is read from:
// • the PREVIOUS row (item i not yet used) → 0/1, no reuse
// • the CURRENT row (item i already used) → unbounded, reuse
// Loop direction decides which one you read.
// Mnemonic: DESCENDING = Distinct (each once), ASCENDING = Again (reusable).
// Seed/operator picks the flavor:
// count → dp[0]=1, dp[j] += dp[j-w]
// max-value → dp[0]=0, dp[j] = Math.max(dp[j], dp[j-w] + v)
// -----

// 2B. KNAPSACK 0/1 – each item at most once
int[] dp = new int[target + 1];
dp[0] = 1; // 1 way to make 0: take nothing
for (int i = 0; i < nums.length; i++)
    for (int j = target; j >= nums[i]; j--) // ↓ DESCENDING
        dp[j] += dp[j - nums[i]];
// ^^^^^^^^^^^^^ j-nums[i] < j, not yet touched THIS pass
// → still "previous row" → item i unused → no reuse

// 2C. KNAPSACK UNBOUNDED – each item reusable
int[] dp = new int[target + 1];
dp[0] = 1;
for (int i = 0; i < nums.length; i++)
    for (int j = nums[i]; j <= target; j++) // ↑ ASCENDING
        dp[j] += dp[j - nums[i]];
// ^^^^^^^^^^^^^ j-nums[i] < j, ALREADY updated THIS pass
// → "current row" → item i can reappear → reuse

// 3. 2D SEQUENCE – two strings/arrays; i indexes one, j indexes the other
// MENTAL MODEL: compare the last char of each prefix – match consumes both, mismatch drops one side.
// WHEN: "compare two strings/arrays" (LCS, edit distance, subsequence count)
int[][] dp = new int[m + 1][n + 1];
for (int i = 1; i <= m; i++) {
    for (int j = 1; j <= n; j++) {
        if (match(i, j)) {
            dp[i][j] = dp[i-1][j-1] + val;
        } else {
            dp[i][j] = combine(dp[i-1][j], dp[i][j-1]);
        }
    }
}
}
```

```
// 4. INTERVAL DP - i goes RIGHT to LEFT; j goes i+1 to end
// "shorter comes first": when computing dp[i][j], dp[i+1][*] is already done
// MENTAL MODEL: build answers for short ranges first, then combine them into longer ranges.
// WHEN: "palindrome substring/subseq", "merge/burst in a range", split-point problems
int[][] dp = new int[n][n];
for (int i = 0; i < n; i++) dp[i][i] = base; // length-1 intervals
for (int i = n - 1; i >= 0; i--) { // ← right-to-left
    for (int j = i + 1; j < n; j++) {
        dp[i][j] = f(dp[i+1][j-1], dp[i+1][j], dp[i][j-1]);
    }
}

// 5. GRID DP - 4-directional neighbors
// MENTAL MODEL: each cell's answer accumulates from the cells you could have arrived from.
// WHEN: "paths / min-cost in a grid moving right+down", "largest square"
int[] dr = {1,-1,0,0}, dc = {0,0,1,-1}; // DOWN UP RIGHT LEFT
// Standard grid (dag, only right/down):
int[][] dp = new int[rows][cols];
for (int i = 0; i < rows; i++)
    for (int j = 0; j < cols; j++)
        dp[i][j] = grid[i][j] + combine(dp[i-1][j], dp[i][j-1]);

// 6. STATE MACHINE - named states updated simultaneously each step
// MENTAL MODEL: track the best value in each named situation; each day every state updates from yesterday's states.
// WHEN: "buy/sell/hold", "limited transactions", "cooldown/fee"
int cash = 0, hold = -prices[0];
for (int i = 1; i < prices.length; i++) {
    cash = Math.max(cash, hold + prices[i]);
    hold = Math.max(hold, ??? - prices[i]); // ??? depends on problem
}
```

Knapsack Decision Tree

```
Want count or min/max?
├─ Count (dp[j] += dp[j - num])
│   ├── 0/1 (each item once):    j descending → #416, #494
│   └─ Unbounded (reuse):        j ascending  → #518
│       └─ Ordered (permut):    target outer, nums inner → #377
└─ Min/Max (dp[j] = min/max(...))
    ├── 0/1 minimize:            j descending → #474 (maximize)
    └─ Unbounded minimize:      j ascending  → #322
```

Part 1 — Linear 1D DP

#70 Climbing Stairs

Description: Each step you can climb 1 or 2 steps. How many distinct ways to reach step n ?

Intuition: the last move was a 1-step or a 2-step, so ways to reach n = ways to $n-1$ + ways to $n-2$.

```
class Solution {
    public int climbStairs(int n) {
        if (n <= 2) return n;
        int[] dp = new int[n + 1];
        dp[1] = 1; dp[2] = 2;
        for (int i = 3; i <= n; i++) dp[i] = dp[i-1] + dp[i-2];
        return dp[n];
    }
}
```

Time $O(n)$ | **Space** $O(n)$

#198 House Robber

Description: Rob houses on a line — can't rob two adjacent. Maximize total.

Intuition: at each house, either skip it (keep $dp[i-1]$) or rob it ($dp[i-2] + \text{value}$, since $i-1$ is now off-limits).

```
class Solution {
    public int rob(int[] nums) {
        int n = nums.length;
        if (n == 1) return nums[0];
        int[] dp = new int[n];
        dp[0] = nums[0]; dp[1] = Math.max(nums[0], nums[1]);
        for (int i = 2; i < n; i++)
            dp[i] = Math.max(dp[i-1], dp[i-2] + nums[i]);
        return dp[n-1];
    }
}
```

Time $O(n)$ | **Space** $O(n)$

#213 House Robber II

Description: Houses arranged in a circle — first and last are adjacent. Can't rob both.

Key: run House Robber on $[0, n-2]$ and $[1, n-1]$; take the max. Two passes, exact same helper.

Intuition: breaking the circle means either the first house is excluded or the last is — solve both lines and keep the better.

```
class Solution {
    public int rob(int[] nums) {
        int n = nums.length;
        if (n == 1) return nums[0];
        return Math.max(rob(nums, 0, n - 2), rob(nums, 1, n - 1));
    }
    private int rob(int[] nums, int i, int j) {
        int prev2 = 0, prev1 = 0;
        for (int k = i; k <= j; k++) {
            int current = Math.max(prev1, prev2 + nums[k]);
            prev2 = prev1;
            prev1 = current;
        }
        return prev1;
    }
}
```

Time $O(n)$ | **Space** $O(1)$

#300 Longest Increasing Subsequence

Description: Length of the longest strictly increasing subsequence.

Template: Look-back 1D — $dp[i]$ depends on all $dp[j]$ where $j < i$ and $nums[j] < nums[i]$.

Intuition: the best subsequence ending at i extends the longest earlier one whose last value is still smaller.

```

class Solution {
    public int lengthOfLIS(int[] nums) {
        int n = nums.length;
        int[] dp = new int[n];
        Arrays.fill(dp, 1);
        int max = 1;
        for (int i = 1; i < n; i++) {
            for (int j = 0; j < i; j++) {
                if (nums[j] < nums[i]) dp[i] = Math.max(dp[i], dp[j] + 1);
            }
            max = Math.max(max, dp[i]);
        }
        return max;
    }
}

```

Time $O(n^2)$ | **Space** $O(n)$

#139 Word Break

Description: Can `s` be segmented into a sequence of dictionary words?

Template: Look-back 1D — `dp[i]` is true if some `dp[j]` is true and `s[j..i)` is a word.

Intuition: a prefix is breakable if some earlier breakable cut leaves a dictionary word as the final piece.

```

class Solution {
    public boolean wordBreak(String s, List<String> wordDict) {
        Set<String> words = new HashSet<>(wordDict);
        int n = s.length();
        boolean[] dp = new boolean[n + 1];
        dp[0] = true;
        for (int i = 1; i <= n; i++) {
            for (int j = 0; j < i; j++) {
                if (dp[j] && words.contains(s.substring(j, i))) {
                    dp[i] = true;
                    break;
                }
            }
        }
        return dp[n];
    }
}

```

Time $O(n^2 \cdot L)$ where L = substring length | **Space** $O(n)$

198 vs 300 vs 139 — Look-back patterns

	#198 House Robber	#300 LIS	#139 Word Break
<code>dp[i]</code> means:	max rob ending at <code>i</code>	LIS ending at <code>i</code>	can break <code>s[0..i)</code>
<code>j</code> range:	<code>j = i-1, i-2</code> (fixed)	<code>j = 0..i-1</code> (all)	<code>j = 0..i-1</code> (all)
condition on <code>j</code> :	none (always)	<code>nums[j] < nums[i]</code>	<code>dp[j] && word in dict</code>
update:	<code>max(dp[i-1], dp[i-2]+x)</code>	<code>max(dp[i], dp[j]+1)</code>	<code>dp[i] = true; break</code>
answer:	<code>dp[n-1]</code>	<code>max(dp)</code>	<code>dp[n]</code>

Part 2 — Knapsack DP

Canonical Templates

```
// — 0/1 KNAPSACK — each item at most once
int[] dp = new int[target + 1];
dp[0] = 1;
for (int i = 0; i < nums.length; i++) {
    for (int j = target; j >= nums[i]; j--)    // DESCENDING: sees OLD dp (before item i)
        dp[j] += dp[j - nums[i]];
}

// — UNBOUNDED KNAPSACK — each item reusable
int[] dp = new int[target + 1];
dp[0] = 1;
for (int i = 0; i < nums.length; i++) {
    for (int j = nums[i]; j <= target; j++)    // ASCENDING: sees NEW dp (after item i already used)
        dp[j] += dp[j - nums[i]];
}

// — PERMUTATION COUNT — order matters (Combination Sum IV)
int[] dp = new int[target + 1];
dp[0] = 1;
for (int j = 1; j <= target; j++) {          // OUTER: target
    for (int num : nums) {                  // INNER: try all nums (not fixing order)
        if (j >= num) dp[j] += dp[j - num];
    }
}
```

Why descending for 0/1:

`dp[j - nums[i]]` is read **before** `dp[j]` is updated ($j > j - \text{nums}[i]$). So it always sees the dp state from before item `i` was considered — no item is used twice.

Why ascending for unbounded:

`dp[j - nums[i]]` was already updated in this same pass ($j - \text{nums}[i] < j$, processed earlier). This allows item `i` to be picked multiple times.

2D anchor: both loops are `dp[i][j] = dp[i-1][j] + dp[i-?][j-w]` collapsed to 1D — descending keeps `dp[j-w]` on the *previous* row (item `i` unused), ascending pulls it from the *current* row (item `i` reused).

Mnemonic: DESCENDING = **D**istinct (each item once), ASCENDING = **A**gain (reusable).

Flavor = seed + operator (same skeleton):

- count → `dp[0]=1` , `dp[j] += dp[j-w]`
- feasibility → `dp[0]=true` , `dp[j] = dp[j] || dp[j-w]`
- max-value → `dp[0]=0` , `dp[j] = Math.max(dp[j], dp[j-w] + v)`

#416 Partition Equal Subset Sum

Description: Can the array be partitioned into two subsets with equal sum?

Key: equivalent to: can we pick a subset summing to `total/2` ?

```

class Solution {
    public boolean canPartition(int[] nums) {
        int sum = Arrays.stream(nums).sum();
        if (sum % 2 != 0) return false;
        int target = sum / 2;
        boolean[] dp = new boolean[target + 1];
        dp[0] = true;
        for (int num : nums) {
            for (int j = target; j >= num; j--)    // 0/1: descending
                dp[j] = dp[j] || dp[j - num];
        }
        return dp[target];
    }
}

```

#494 Target Sum

Description: Assign `+` or `-` to each number to reach `target`. How many ways?

Key: assign `+` to subset P and `-` to subset N. Then $P - N = \text{target}$ and $P + N = \text{sum}$. So $P = (\text{sum} + \text{target}) / 2$. Count subsets summing to P.

```

class Solution {
    public int findTargetSumWays(int[] nums, int target) {
        int sum = Arrays.stream(nums).sum();
        if ((sum + target) % 2 != 0 || Math.abs(target) > sum) return 0;
        int t = (sum + target) / 2;
        int[] dp = new int[t + 1];
        dp[0] = 1;
        for (int num : nums) {
            for (int j = t; j >= num; j--)    // 0/1: descending
                dp[j] += dp[j - num];
        }
        return dp[t];
    }
}

```

#474 Ones and Zeroes

Description: Given strings of '0'/'1', find the largest subset using at most `m` zeros and `n` ones.

Key: 2D 0/1 knapsack — two capacity dimensions. Both must iterate descending.

```

class Solution {
    public int findMaxForm(String[] strs, int m, int n) {
        int[][] dp = new int[m + 1][n + 1];
        for (String s : strs) {
            int zeros = 0, ones = 0;
            for (char c : s.toCharArray()) { if (c == '0') zeros++; else ones++; }
            for (int i = m; i >= zeros; i--)    // 0/1: descending (both dims)
                for (int j = n; j >= ones; j--)
                    dp[i][j] = Math.max(dp[i][j], dp[i - zeros][j - ones] + 1);
        }
        return dp[m][n];
    }
}

```

#322 Coin Change

Description: Minimum number of coins to make `amount`. Coins can be reused.

Key: unbounded knapsack (ascending), but minimize instead of count — seed with INF, take min.

```

class Solution {
    public int coinChange(int[] coins, int amount) {
        int[] dp = new int[amount + 1];
        Arrays.fill(dp, amount + 1);          // impossible sentinel
        dp[0] = 0;
        for (int coin : coins) {
            for (int j = coin; j <= amount; j++)    // unbounded: ascending
                dp[j] = Math.min(dp[j], dp[j - coin] + 1);
        }
        return dp[amount] > amount ? -1 : dp[amount];
    }
}

```

#518 Coin Change II

Description: Number of distinct combinations (order doesn't matter) to make `amount` .

Key: unbounded knapsack (ascending), count variant — combinations, not permutations.

```

class Solution {
    public int change(int amount, int[] coins) {
        int[] dp = new int[amount + 1];
        dp[0] = 1;
        for (int coin : coins) {
            for (int j = coin; j <= amount; j++)    // unbounded: ascending
                dp[j] += dp[j - coin];
        }
        return dp[amount];
    }
}

```

#377 Combination Sum IV

Description: Number of ordered sequences (order matters) that sum to `target` .

Key: permutation count — swap loop order: target outer, nums inner. This tries all nums for each partial sum, so different orderings are counted separately.

```

class Solution {
    public int combinationSum4(int[] nums, int target) {
        int[] dp = new int[target + 1];
        dp[0] = 1;
        for (int j = 1; j <= target; j++) {        // outer: target value
            for (int num : nums) {                // inner: all numbers
                if (j >= num) dp[j] += dp[j - num];
            }
        }
        return dp[target];
    }
}

```

518 vs 377 — Side by Side

	#518 Coin Change II	#377 Combination Sum IV
Loop order:	coins outer, j inner	j outer, nums inner
Counts:	combinations (sets)	permutations (sequences)
Example (1,2-3):	{1,2} and {1,1,1} = 2	{1,2},{2,1},{1,1,1} = 3
Why:	fixing coin order = no dup	re-trying all nums each j = all orders

Part 3 — 2D Sequence DP

Canonical Template

```
// i indexes string/array 1 (1-indexed), j indexes string/array 2
int[][] dp = new int[m + 1][n + 1];
// Init dp[0][*] and dp[*][0] based on problem
for (int i = 1; i <= m; i++) {
    for (int j = 1; j <= n; j++) {
        if (s1.charAt(i-1) == s2.charAt(j-1)) {
            dp[i][j] = dp[i-1][j-1] + 1;          // characters match
        } else {
            dp[i][j] = combine(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]);
        }
    }
}
```

#1143 Longest Common Subsequence

Description: Length of the longest subsequence present in both strings.

Intuition: if the last chars match, they're part of the LCS (+1 on the diagonal); otherwise drop one char from whichever string and keep the better.

```
class Solution {
    public int longestCommonSubsequence(String text1, String text2) {
        int m = text1.length(), n = text2.length();
        int[][] dp = new int[m + 1][n + 1];
        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (text1.charAt(i-1) == text2.charAt(j-1)) {
                    dp[i][j] = dp[i-1][j-1] + 1;
                } else {
                    dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1]);
                }
            }
        }
        return dp[m][n];
    }
}
```

Time $O(m \cdot n)$ | **Space** $O(m \cdot n)$

#72 Edit Distance

Description: Minimum operations (insert, delete, replace) to convert `word1` to `word2`.

Init: $dp[i][0] = i$ (delete all), $dp[0][j] = j$ (insert all).

Intuition: matched chars cost nothing (diagonal); otherwise take the cheapest of replace/delete/insert and add 1.

```

class Solution {
    public int minDistance(String word1, String word2) {
        int m = word1.length(), n = word2.length();
        int[][] dp = new int[m + 1][n + 1];
        for (int i = 0; i <= m; i++) dp[i][0] = i;
        for (int j = 0; j <= n; j++) dp[0][j] = j;
        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (word1.charAt(i-1) == word2.charAt(j-1)) {
                    dp[i][j] = dp[i-1][j-1]; // match: no cost
                } else {
                    dp[i][j] = 1 + Math.min(dp[i-1][j-1], // replace
                                           Math.min(dp[i-1][j], // delete
                                                    dp[i][j-1])); // insert
                }
            }
        }
        return dp[m][n];
    }
}

```

Time $O(m \cdot n)$ | **Space** $O(m \cdot n)$

#115 Distinct Subsequences

Description: Number of ways `s` contains `t` as a subsequence.

Init: `dp[i][0] = 1` (empty `t` matched by any `s` prefix).

Intuition: you can always skip the current char of `s`; if it matches the current char of `t`, you may also use it to advance `t`.

```

class Solution {
    public int numDistinct(String s, String t) {
        int m = s.length(), n = t.length();
        int[][] dp = new int[m + 1][n + 1];
        for (int i = 0; i <= m; i++) dp[i][0] = 1;
        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                dp[i][j] = dp[i-1][j]; // always: skip s[i]
                if (s.charAt(i-1) == t.charAt(j-1))
                    dp[i][j] += dp[i-1][j-1]; // also: use s[i] to match t[j]
            }
        }
        return dp[m][n];
    }
}

```

Time $O(m \cdot n)$ | **Space** $O(m \cdot n)$

1143 vs 72 vs 115 — Transition Cheat Sheet

	#1143 LCS	#72 Edit Distance	#115 Distinct Subseq
Match (<code>s[i]==t[j]</code>):	<code>dp[i-1][j-1]+1</code>	<code>dp[i-1][j-1]</code>	<code>dp[i-1][j]+dp[i-1][j-1]</code>
No match:	<code>max(up, left)</code>	<code>1+min(diag, up, left)</code>	<code>dp[i-1][j]</code> (skip <code>s[i]</code>)
Init <code>dp[0][*]</code> :	0	<code>j</code> (insert <code>j</code> chars)	0
Init <code>dp[*][0]</code> :	0	<code>i</code> (delete <code>i</code> chars)	1 (empty <code>t</code> always matches)

Part 4 — Interval DP

Canonical Template

Rule: `i` goes right-to-left (ensures shorter/inner intervals are computed first). `j` goes left-to-right from `i+1`. When computing `dp[i][j]`, all `dp[i'][j']` with `i' > i` or `j' < j` are already done.

```
int[][] dp = new int[n][n];
for (int i = 0; i < n; i++) dp[i][i] = base_case; // single elements
for (int i = n - 1; i >= 0; i--) { // right-to-left
    for (int j = i + 1; j < n; j++) { // left-to-right from i+1
        // dp[i][j] can safely use dp[i+1][j], dp[i][j-1], dp[i+1][j-1]
        dp[i][j] = ...;
    }
}
```

Alternative (length-based): Equivalent, sometimes clearer for k-partition problems:

```
for (int len = 2; len <= n; len++) { // outer: length of interval
    for (int i = 0; i + len - 1 < n; i++) { // inner: left endpoint
        int j = i + len - 1;
        dp[i][j] = ...;
    }
}
```

#647 Palindromic Substrings

Description: Count all palindromic substrings of `s`.

Intuition: `s[i..j]` is a palindrome when its two ends match AND the inside `s[i+1..j-1]` already is.

```
class Solution {
    public int countSubstrings(String s) {
        int n = s.length(), count = 0;
        boolean[][] dp = new boolean[n][n];
        for (int i = n - 1; i >= 0; i--) {
            for (int j = i; j < n; j++) {
                dp[i][j] = s.charAt(i) == s.charAt(j)
                    && (j - i < 2 || dp[i+1][j-1]); // ends match && inner is palindrome
                if (dp[i][j]) count++;
            }
        }
        return count;
    }
}
```

Time $O(n^2)$ | **Space** $O(n^2)$

#516 Longest Palindromic Subsequence

Description: Length of the longest subsequence of `s` that is a palindrome.

Intuition: matching ends add 2 to the inner range's best; otherwise drop one end and keep the larger.


```

class Solution {
    public int longestPalindromeSubseq(String s) {
        int n = s.length();
        int[][] dp = new int[n][n];
        for (int i = 0; i < n; i++) dp[i][i] = 1;
        for (int i = n - 1; i >= 0; i--) {
            for (int j = i + 1; j < n; j++) {
                if (s.charAt(i) == s.charAt(j)) {
                    dp[i][j] = dp[i+1][j-1] + 2;
                } else {
                    dp[i][j] = Math.max(dp[i+1][j], dp[i][j-1]);
                }
            }
        }
        return dp[0][n-1];
    }
}

```

Time $O(n^2)$ | **Space** $O(n^2)$

#312 Burst Balloons

Description: Burst all balloons to maximize coins. Coins from bursting balloon `k` between open boundaries `i` and `j` = `balloons[i] * balloons[k] * balloons[j]`.

Key insight: think of `k` as the **last** balloon burst in range `(i, j)` (open interval). Adding sentinel `1` s at both ends simplifies boundary cases.

Template: length-based outer loop; inner loop over last-burst position `k`.

```

class Solution {
    public int maxCoins(int[] nums) {
        int n = nums.length;
        int[] b = new int[n + 2];
        b[0] = b[n + 1] = 1;
        for (int i = 0; i < n; i++) b[i + 1] = nums[i];
        int size = n + 2;
        int[][] dp = new int[size][size];
        for (int len = 2; len < size; len++) {
            for (int i = 0; i + len < size; i++) {
                int j = i + len;
                for (int k = i + 1; k < j; k++) { // k = last balloon burst in (i, j)
                    dp[i][j] = Math.max(dp[i][j],
                        dp[i][k] + b[i] * b[k] * b[j] + dp[k][j]);
                }
            }
        }
        return dp[0][n + 1];
    }
}

```

Time $O(n^3)$ | **Space** $O(n^2)$

Part 5 — Grid DP

Canonical Template

```
int[] dr = {1,-1,0,0}, dc = {0,0,1,-1};    // DOWN UP RIGHT LEFT

// Standard grid (only move right/down – DAG, no cycle):
int[][] dp = new int[rows][cols];
// initialize first row and first column
for (int i = 0; i < rows; i++)
    for (int j = 0; j < cols; j++)
        dp[i][j] = grid[i][j] + f(dp[i-1][j], dp[i][j-1]);

// All-direction grid (memoized DFS for increasing/decreasing paths):
int[][] memo = new int[rows][cols];
for (int i = 0; i < rows; i++)
    for (int j = 0; j < cols; j++)
        dfs(grid, i, j, memo, dr, dc);
```

#62 Unique Paths

Description: Number of paths from top-left to bottom-right, moving only right or down.

Init: first row and column are all 1 (only one way to reach any edge cell).

Intuition: you reach a cell only from above or from the left, so its path count is the sum of those two.

```
class Solution {
    public int uniquePaths(int m, int n) {
        int[][] dp = new int[m][n];
        for (int i = 0; i < m; i++) dp[i][0] = 1;
        for (int j = 0; j < n; j++) dp[0][j] = 1;
        for (int i = 1; i < m; i++)
            for (int j = 1; j < n; j++)
                dp[i][j] = dp[i-1][j] + dp[i][j-1];
        return dp[m-1][n-1];
    }
}
```

Time $O(m \cdot n)$ | **Space** $O(m \cdot n)$

#64 Minimum Path Sum

Description: Minimum sum path from top-left to bottom-right, moving only right or down.

Init: first row/column are cumulative sums (no choice on edges).

Intuition: the cheapest way into a cell is its own value plus the cheaper of the cell above or to its left.

```
class Solution {
    public int minPathSum(int[][] grid) {
        int m = grid.length, n = grid[0].length;
        int[][] dp = new int[m][n];
        dp[0][0] = grid[0][0];
        for (int i = 1; i < m; i++) dp[i][0] = dp[i-1][0] + grid[i][0];
        for (int j = 1; j < n; j++) dp[0][j] = dp[0][j-1] + grid[0][j];
        for (int i = 1; i < m; i++)
            for (int j = 1; j < n; j++)
                dp[i][j] = grid[i][j] + Math.min(dp[i-1][j], dp[i][j-1]);
        return dp[m-1][n-1];
    }
}
```

Time $O(m \cdot n)$ | **Space** $O(m \cdot n)$

#221 Maximal Square

Description: Find the largest all-1 square in a binary matrix. Return its area.

Key insight: $dp[i][j]$ = side length of the largest all-1 square with its bottom-right corner at (i, j) . Limited by the minimum of up, left, and diagonal neighbors.

```
class Solution {
    public int maximalSquare(char[][] matrix) {
        int m = matrix.length, n = matrix[0].length, maxSide = 0;
        int[][] dp = new int[m][n];
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (matrix[i][j] == '1') {
                    dp[i][j] = (i == 0 || j == 0) ? 1
                        : Math.min(dp[i-1][j], Math.min(dp[i][j-1], dp[i-1][j-1])) + 1;
                    maxSide = Math.max(maxSide, dp[i][j]);
                }
            }
        }
        return maxSide * maxSide;
    }
}
```

Time $O(m \cdot n)$ | **Space** $O(m \cdot n)$

#329 Longest Increasing Path in a Matrix

Description: Find the longest strictly increasing path. Can move in all 4 directions.

Template: DFS + memoization (grid has ordering from values, so no cycle in the recursion DAG).

Intuition: strictly increasing values forbid revisiting, so each cell's longest path is 1 + the best of its larger neighbors — cache it.

```
class Solution {
    int[] dr = {1,-1,0,0}, dc = {0,0,1,-1};

    public int longestIncreasingPath(int[][] matrix) {
        int m = matrix.length, n = matrix[0].length;
        int[][] memo = new int[m][n];
        int max = 0;
        for (int i = 0; i < m; i++)
            for (int j = 0; j < n; j++)
                max = Math.max(max, dfs(matrix, i, j, memo));
        return max;
    }
    private int dfs(int[][] matrix, int i, int j, int[][] memo) {
        if (memo[i][j] != 0) return memo[i][j];
        memo[i][j] = 1;
        for (int dir = 0; dir < 4; dir++) {
            int ni = i + dr[dir], nj = j + dc[dir];
            if (ni >= 0 && ni < matrix.length && nj >= 0 && nj < matrix[0].length
                && matrix[ni][nj] > matrix[i][j])
                memo[i][j] = Math.max(memo[i][j], dfs(matrix, ni, nj, memo) + 1);
        }
        return memo[i][j];
    }
}
```

Time $O(m \cdot n)$ | **Space** $O(m \cdot n)$

Part 6 — State Machine DP (Stock Problems)

Canonical States

```
cash      = max profit when NOT holding (free to buy or idle)
hold      = max profit when HOLDING stock (bought it at some cost)
cooldown  = max profit right after selling (can't buy today)
```

All Five Transitions

	buy	sell	re-buy condition
#121 (1 tx):	hold = max(hold, -price)		no re-buy (ignore cash)
#122 (unlimited):	hold = max(hold, cash - price)		re-buy with full cash
#123 (2 tx):	chain: buy2 uses sell1 cash		4 states chained
#309 (cooldown):	hold = max(hold, cooldown - price)		must wait 1 day
#714 (fee):	hold = max(hold, cash - price)		pay fee on sell

#121 Best Time to Buy and Sell Stock (at most 1 transaction)

Intuition: `hold` is the best profit if you currently own a share bought at the lowest price seen; sell whenever today's price beats that.

```
class Solution {
    public int maxProfit(int[] prices) {
        int cash = 0, hold = -prices[0];
        for (int i = 1; i < prices.length; i++) {
            cash = Math.max(cash, hold + prices[i]);
            hold = Math.max(hold, -prices[i]); // no cash reinvestment (1 buy)
        }
        return cash;
    }
}
```

Time $O(n)$ | **Space** $O(1)$

#122 Best Time to Buy and Sell Stock II (unlimited transactions)

Key difference from #121: `hold = max(hold, cash - prices[i])` — can reinvest all prior profits.

Intuition: with unlimited trades, buying can fund itself from accumulated `cash`, so capture every upward move.

```
class Solution {
    public int maxProfit(int[] prices) {
        int cash = 0, hold = -prices[0];
        for (int i = 1; i < prices.length; i++) {
            cash = Math.max(cash, hold + prices[i]);
            hold = Math.max(hold, cash - prices[i]); // reinvest cash (unlimited buys)
        }
        return cash;
    }
}
```

Time $O(n)$ | **Space** $O(1)$

#123 Best Time to Buy and Sell Stock III (at most 2 transactions)

Key: 4 named states chained in order: `buy1` → `sell1` → `buy2` → `sell2`.

Intuition: the second buy can only spend profit left after the first sell, so chain the states in trade order.

```

class Solution {
    public int maxProfit(int[] prices) {
        int buy1 = -prices[0], sell1 = 0, buy2 = -prices[0], sell2 = 0;
        for (int i = 1; i < prices.length; i++) {
            buy1 = Math.max(buy1, -prices[i]);
            sell1 = Math.max(sell1, buy1 + prices[i]);
            buy2 = Math.max(buy2, sell1 - prices[i]);
            sell2 = Math.max(sell2, buy2 + prices[i]);
        }
        return sell2;
    }
}

```

Time $O(n)$ | **Space** $O(1)$

#309 Best Time to Buy and Sell Stock with Cooldown

Description: After selling, must wait 1 day before buying again.

Key: `hold` can only use `cooldown` (yesterday's `cash`), not today's `cash`. Save `prevCash` before updating.

Intuition: the cooldown forces buying to draw from cash that's at least one day old, so carry yesterday's cash forward.

```

class Solution {
    public int maxProfit(int[] prices) {
        int cash = 0, hold = -prices[0], cooldown = 0;
        for (int i = 1; i < prices.length; i++) {
            int prevCash = cash;
            cash = Math.max(cash, hold + prices[i]);
            hold = Math.max(hold, cooldown - prices[i]); // buy only from cooldown state
            cooldown = prevCash; // yesterday's cash
        }
        return cash;
    }
}

```

Time $O(n)$ | **Space** $O(1)$

#714 Best Time to Buy and Sell Stock with Transaction Fee

Description: Pay a fee per transaction (on sell). Unlimited transactions.

Key: same as #122 but subtract `fee` when selling.

Intuition: the fee just shrinks every sale, so a trade is only worth making when the gain clears the fee.

```

class Solution {
    public int maxProfit(int[] prices, int fee) {
        int cash = 0, hold = -prices[0];
        for (int i = 1; i < prices.length; i++) {
            cash = Math.max(cash, hold + prices[i] - fee); // pay fee on sell
            hold = Math.max(hold, cash - prices[i]);
        }
        return cash;
    }
}

```

Time $O(n)$ | **Space** $O(1)$

Stock Problems — Differences at a Glance

Problem	hold update	cash update	Extra state
#121	<code>max(hold, -price)</code>	<code>max(cash, hold+price)</code>	none
#122	<code>max(hold, cash-price)</code>	<code>max(cash, hold+price)</code>	none
#123	4 variables, chained	—	<code>sell1</code> feeds <code>buy2</code>
#309	<code>max(hold, cooldown-price)</code>	<code>max(cash, hold+price)</code>	<code>cooldown = prevCash</code>
#714	<code>max(hold, cash-price)</code>	<code>max(cash, hold+price-fee)</code>	none

Template Chooser

Signal in problem	Template
<code>dp[i]</code> depends on <code>dp[i-1]</code> , <code>dp[i-2]</code>	Linear 1D — fixed lookback
<code>dp[i]</code> depends on all <code>dp[j]</code> where <code>j < i</code>	Look-back 1D — $O(n^2)$ inner loop
Each item used at most once, sum target	Knapsack 0/1 — <code>j</code> descending
Items reusable, sum target, order irrelevant	Knapsack Unbounded — <code>j</code> ascending
Items reusable, order matters (permutations)	Permutation count — target outer
Two strings/arrays, compare characters	2D Sequence DP
Palindrome, substring merge, balloon burst	Interval DP — <code>i</code> right-to-left
Grid, only right/down movement	Grid DP — cumulative from edges
Grid, all 4 directions, monotone constraint	DFS + memo (no cycle in DAG)
Buy/sell/hold decision changes day to day	State Machine — named states

#746 Min Cost Climbing Stairs

Description: Each step has a cost. You can start from index 0 or 1, and from each step you can climb 1 or 2 steps. Return the minimum cost to reach the top (past the last index).

Algorithm: 1D DP. `dp[i]` = min cost to reach step `i` . `dp[0] = dp[1] = 0` (can start for free). For `i >= 2` : `dp[i] = min(dp[i-1] + cost[i-1], dp[i-2] + cost[i-2])` .

```
class Solution {
    public int minCostClimbingStairs(int[] cost) {
        int n = cost.length;
        int[] dp = new int[n + 1];
        for (int i = 2; i <= n; i++)
            dp[i] = Math.min(dp[i-1] + cost[i-1], dp[i-2] + cost[i-2]); // ← two sources
        return dp[n];
    }
}
```

Time $O(n)$ | **Space** $O(n)$

#91 Decode Ways

Description: A string of digits can be decoded: '1'-'9' → single letter, "10"-"26" → two-digit letter. Count total decoding ways.

Variation: Two sources at each position. If the current digit is non-zero, it can be decoded alone (`dp[i] += dp[i-1]`). If the previous two digits form a valid two-letter code (10-26), add `dp[i-2]` .

```

class Solution {
    public int numDecodings(String s) {
        if (s.charAt(0) == '0') return 0;
        int n = s.length();
        int[] dp = new int[n + 1];
        dp[0] = 1; dp[1] = 1;
        for (int i = 2; i <= n; i++) {
            int oneDigit = s.charAt(i-1) - '0';
            int twoDigit = Integer.parseInt(s.substring(i-2, i));
            if (oneDigit != 0) dp[i] += dp[i-1]; // ← VARIATION: single digit (1-9)
            if (twoDigit >= 10 && twoDigit <= 26) dp[i] += dp[i-2]; // ← VARIATION: two digits (10-26)
        }
        return dp[n];
    }
}

```

Time $O(n)$ | **Space** $O(n)$

#279 Perfect Squares

Description: Return the minimum number of perfect squares (1, 4, 9, 16, ...) that sum to n .

Algorithm: Unbounded knapsack. $dp[i]$ = min squares to reach sum i . For each i , try all squares $j*j \leq i$: $dp[i] = \min(dp[i], dp[i - j*j] + 1)$.

```

class Solution {
    public int numSquares(int n) {
        int[] dp = new int[n + 1];
        Arrays.fill(dp, n + 1);
        dp[0] = 0;
        for (int i = 1; i <= n; i++)
            for (int j = 1; j * j <= i; j++) // ← VARIATION: iterate all squares ≤ i
                dp[i] = Math.min(dp[i], dp[i - j*j] + 1);
        return dp[n];
    }
}

```

Time $O(n\sqrt{n})$ | **Space** $O(n)$

#96 Unique Binary Search Trees

Description: Return the number of structurally unique BSTs that store values 1 to n .

Variation: Catalan number recurrence. $dp[i]$ = number of unique BSTs with i nodes. When root = j , left subtree has $j-1$ nodes, right subtree has $i-j$ nodes: $dp[i] += dp[j-1] * dp[i-j]$.

```

class Solution {
    public int numTrees(int n) {
        int[] dp = new int[n + 1];
        dp[0] = dp[1] = 1;
        for (int i = 2; i <= n; i++)
            for (int j = 1; j <= i; j++)
                dp[i] += dp[j-1] * dp[i-j]; // ← VARIATION: Catalan: root=j, left=j-1, right=i-j
        return dp[n];
    }
}

```

Time $O(n^2)$ | **Space** $O(n)$

#63 Unique Paths II

Description: A robot moves from top-left to bottom-right in an $m \times n$ grid with obstacles. Count unique paths. Cells with `obstacleGrid[i][j] == 1` are blocked.

Variation: Same as #62 (Unique Paths) but skip cells with obstacles: `dp[i][j] = 0` if blocked, else `dp[i-1][j] + dp[i][j-1]`.

```
class Solution {
    public int uniquePathsWithObstacles(int[][] obstacleGrid) {
        int m = obstacleGrid.length, n = obstacleGrid[0].length;
        int[][] dp = new int[m][n];
        for (int i = 0; i < m && obstacleGrid[i][0] == 0; i++) dp[i][0] = 1; // ← VARIATION: stop at obstacle
        for (int j = 0; j < n && obstacleGrid[0][j] == 0; j++) dp[0][j] = 1;
        for (int i = 1; i < m; i++)
            for (int j = 1; j < n; j++)
                if (obstacleGrid[i][j] == 0) // ← VARIATION: skip blocked cells
                    dp[i][j] = dp[i-1][j] + dp[i][j-1];
        return dp[m-1][n-1];
    }
}
```

Time $O(m \cdot n)$ | **Space** $O(m \cdot n)$

#583 Delete Operation for Two Strings

Description: Return the minimum number of deletions to make `word1` and `word2` equal.

Variation: Reduce to LCS. Minimum deletions = `len(word1) + len(word2) - 2 * LCS(word1, word2)`. Compute LCS using standard 2D DP.

```
class Solution {
    public int minDistance(String word1, String word2) {
        int m = word1.length(), n = word2.length();
        int[][] dp = new int[m+1][n+1]; // dp[i][j] = LCS length
        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (word1.charAt(i-1) == word2.charAt(j-1)) {
                    dp[i][j] = dp[i-1][j-1] + 1;
                } else {
                    dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1]);
                }
            }
        }
        return m + n - 2 * dp[m][n]; // ← VARIATION: reduce to LCS
    }
}
```

Time $O(m \cdot n)$ | **Space** $O(m \cdot n)$

#368 Largest Divisible Subset

Description: Find the largest subset of distinct positive integers such that for every pair (a, b) , either $a \% b == 0$ or $b \% a == 0$.

Variation: Sort first, then apply LIS-style DP. `dp[i]` = size of largest divisible subset ending at `nums[i]`. If `nums[i] % nums[j] == 0`, then `dp[i] = max(dp[i], dp[j] + 1)`. Track parent indices to reconstruct the subset.


```

class Solution {
    public List<Integer> largestDivisibleSubset(int[] nums) {
        Arrays.sort(nums); // ← VARIATION: sort required for divisibility chaining
        int n = nums.length;
        int[] dp = new int[n], parent = new int[n];
        Arrays.fill(dp, 1);
        for (int i = 0; i < n; i++) parent[i] = i;
        int maxLen = 1, maxIdx = 0;
        for (int i = 1; i < n; i++) {
            for (int j = 0; j < i; j++) {
                if (nums[i] % nums[j] == 0 && dp[j] + 1 > dp[i]) { // ← VARIATION: divisibility check
                    dp[i] = dp[j] + 1;
                    parent[i] = j;
                }
            }
            if (dp[i] > maxLen) { maxLen = dp[i]; maxIdx = i; }
        }
        List<Integer> result = new ArrayList<>();
        int i = maxIdx;
        while (parent[i] != i) { result.add(nums[i]); i = parent[i]; }
        result.add(nums[i]);
        Collections.reverse(result);
        return result;
    }
}

```

Time $O(n^2)$ | **Space** $O(n)$

#931 Minimum Falling Path Sum

Description: Given an $n \times n$ matrix, return the minimum sum of a falling path. A falling path starts at any element in the first row and chooses the element directly below or diagonally adjacent in each row.

Variation: Grid DP with three sources from the row above. $dp[i][j] = matrix[i][j] + \min(dp[i-1][j-1], dp[i-1][j], dp[i-1][j+1])$.

```

class Solution {
    public int minFallingPathSum(int[][] matrix) {
        int n = matrix.length;
        int[][] dp = new int[n][n];
        dp[0] = matrix[0].clone();
        for (int i = 1; i < n; i++) {
            for (int j = 0; j < n; j++) {
                int best = dp[i-1][j];
                if (j > 0) best = Math.min(best, dp[i-1][j-1]); // ← VARIATION: three sources
                if (j < n-1) best = Math.min(best, dp[i-1][j+1]);
                dp[i][j] = matrix[i][j] + best;
            }
        }
        int min = Integer.MAX_VALUE;
        for (int val : dp[n-1]) min = Math.min(min, val);
        return min;
    }
}

```

Time $O(n^2)$ | **Space** $O(n^2)$

#10 Regular Expression Matching

Description: Implement regex matching with `.` (any single char) and `*` (zero or more of the preceding element). Match must cover the entire input string. **Variation:** 2D DP. $dp[i][j]$ = whether $s[0..i)$ matches $p[0..j)$. The `*` has two cases: zero occurrences ($dp[i][j-2]$) or one-more occurrence when the preceding pattern char matches $s[i-1]$.

```

class Solution {
    public boolean isMatch(String s, String p) {
        int m = s.length(), n = p.length();
        boolean[][] dp = new boolean[m+1][n+1];
        dp[0][0] = true;
        for (int j = 1; j <= n; j++)
            if (p.charAt(j-1) == '*') dp[0][j] = dp[0][j-2]; // ← VARIATION: '*' can erase preceding
        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (p.charAt(j-1) == '*') {
                    dp[i][j] = dp[i][j-2]; // ← VARIATION: zero occurrence
                    if (matches(s, p, i, j-1))
                        dp[i][j] = dp[i][j] || dp[i-1][j]; // ← VARIATION: one more occurrence
                } else if (matches(s, p, i, j)) {
                    dp[i][j] = dp[i-1][j-1];
                }
            }
        }
        return dp[m][n];
    }
    private boolean matches(String s, String p, int i, int j) {
        return p.charAt(j-1) == '.' || s.charAt(i-1) == p.charAt(j-1);
    }
}

```

Time $O(m \cdot n)$ | **Space** $O(m \cdot n)$

#87 Scramble String

Description: Given s_1 and s_2 , return true if s_2 is a scrambled string of s_1 (formed by recursively partitioning into two non-empty substrings and optionally swapping them). **Variation:** memoized recursion over (i_1, i_2, len) . At each length, try every split point both swapped and non-swapped.

```

class Solution {
    private Map<String, Boolean> memo = new HashMap<>();
    public boolean isScramble(String s1, String s2) {
        if (s1.equals(s2)) return true;
        if (s1.length() != s2.length()) return false;
        String key = s1 + "#" + s2;
        if (memo.containsKey(key)) return memo.get(key);
        int n = s1.length();
        int[] count = new int[26];
        for (int i = 0; i < n; i++) { count[s1.charAt(i) - 'a']++; count[s2.charAt(i) - 'a']--; }
        for (int c : count) if (c != 0) { memo.put(key, false); return false; } // char mismatch prune
        for (int i = 1; i < n; i++) {
            // no swap
            if (isScramble(s1.substring(0, i), s2.substring(0, i))
                && isScramble(s1.substring(i), s2.substring(i))) { memo.put(key, true); return true; }
            // swap // ← VARIATION: try swapped partitions
            if (isScramble(s1.substring(0, i), s2.substring(n-i))
                && isScramble(s1.substring(i), s2.substring(0, n-i))) { memo.put(key, true); return true; }
        }
        memo.put(key, false); return false;
    }
}

```

Time $O(n^4)$ | **Space** $O(n^3)$

#1049 Last Stone Weight II

Description: Repeatedly smash the two heaviest stones (result is their difference). Return the smallest possible weight of the remaining stone.

Variation: equivalent to splitting stones into two groups to minimize the difference of their sums. 0/1 knapsack to find the max subset sum $\leq \text{total}/2$.

```

class Solution {
    public int lastStoneWeightII(int[] stones) {
        int total = 0;
        for (int s : stones) total += s;
        int target = total / 2;          // ← VARIATION: subset sum target
        boolean[] dp = new boolean[target + 1];
        dp[0] = true;
        for (int s : stones)
            for (int j = target; j >= s; j--)          // ← 0/1 knapsack: j descending
                dp[j] = dp[j] || dp[j - s];
        for (int j = target; j >= 0; j--)
            if (dp[j]) return total - 2 * j;          // ← VARIATION: minimize |S1-S2|
        return 0;
    }
}

```

Time $O(n \cdot \text{sum})$ | **Space** $O(\text{sum})$

#1312 Minimum Insertion Steps to Make a String Palindrome

Description: Return the minimum number of insertions needed to make a string a palindrome. **Variation:** answer = $n - \text{longest palindromic subsequence (LPS)}$. LPS = LCS of the string and its reverse; or solve directly via interval DP.

```

class Solution {
    public int minInsertions(String s) {
        int n = s.length();
        int[][] dp = new int[n][n];          // dp[i][j] = LPS length in s[i..j]
        for (int i = n - 1; i >= 0; i--) {    // ← interval DP: shorter spans first
            dp[i][i] = 1;
            for (int j = i + 1; j < n; j++) {
                if (s.charAt(i) == s.charAt(j)) {
                    dp[i][j] = dp[i+1][j-1] + 2;
                } else {
                    dp[i][j] = Math.max(dp[i+1][j], dp[i][j-1]);
                }
            }
        }
        return n - dp[0][n-1];              // ← VARIATION: insertions = n - LPS
    }
}

```

Time $O(n^2)$ | **Space** $O(n^2)$

Part 7 — Additional DP Problems

#44 Wildcard Matching

Description: Match string `s` against pattern `p` with `?` (any single char) and `*` (any sequence including empty). **Intuition:** `*` either consumes one more char of `s` (`dp[i-1][j]`) or matches empty (`dp[i][j-1]`); everything else is a 1-to-1 char/ `?` match. **Algorithm:** `dp[i][j] = s[0..i)` matches `p[0..j)`; init `dp[0][j]` true while pattern is all `*`; answer `dp[m][n]`.

```

class Solution {
    public boolean isMatch(String s, String p) {
        int m = s.length(), n = p.length();
        boolean[][] dp = new boolean[m+1][n+1];
        dp[0][0] = true;
        for (int j = 1; j <= n; j++) {
            if (p.charAt(j-1) == '*') {
                dp[0][j] = dp[0][j-1]; // ← VARIATION: '*' matches empty prefix
            }
        }
        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (p.charAt(j-1) == '*') {
                    dp[i][j] = dp[i-1][j] || dp[i][j-1]; // ← VARIATION: '*' consumes char or matches empty
                } else if (p.charAt(j-1) == '?' || s.charAt(i-1) == p.charAt(j-1)) {
                    dp[i][j] = dp[i-1][j-1];
                }
            }
        }
        return dp[m][n];
    }
}

```

Time $O(m \cdot n)$ | **Space** $O(m \cdot n)$

#53 Maximum Subarray

Description: Find the contiguous subarray with the largest sum. **Intuition:** the best sum ending at `i` either extends the previous best or restarts at `nums[i]` (Kadane). **Algorithm:** `current = max sum ending at i = max(nums[i], current + nums[i])`; track global `result`.

```

class Solution {
    public int maxSubArray(int[] nums) {
        int current = nums[0], result = nums[0];
        for (int i = 1; i < nums.length; i++) {
            current = Math.max(nums[i], current + nums[i]); // ← VARIATION: Kadane restart vs extend
            result = Math.max(result, current);
        }
        return result;
    }
}

```

Time $O(n)$ | **Space** $O(1)$

#97 Interleaving String

Description: Return whether `s3` is formed by interleaving `s1` and `s2` while preserving each one's order. **Intuition:** the last char of `s3[0..i+j)` comes from either `s1` or `s2`; reuse the answer for the smaller prefix. **Algorithm:** `dp[i][j] = s1[0..i) + s2[0..j) interleave to s3[0..i+j)`; answer `dp[m][n]`.

```

class Solution {
    public boolean isInterleave(String s1, String s2, String s3) {
        int m = s1.length(), n = s2.length();
        if (m + n != s3.length()) {
            return false;
        }
        boolean[][] dp = new boolean[m+1][n+1];
        dp[0][0] = true;
        for (int i = 1; i <= m; i++) {
            dp[i][0] = dp[i-1][0] && s1.charAt(i-1) == s3.charAt(i-1);
        }
        for (int j = 1; j <= n; j++) {
            dp[0][j] = dp[0][j-1] && s2.charAt(j-1) == s3.charAt(j-1);
        }
        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                dp[i][j] = (dp[i-1][j] && s1.charAt(i-1) == s3.charAt(i+j-1)) // ← VARIATION: take from s1
                    || (dp[i][j-1] && s2.charAt(j-1) == s3.charAt(i+j-1)); // ← VARIATION: take from s2
            }
        }
        return dp[m][n];
    }
}

```

Time $O(m \cdot n)$ | **Space** $O(m \cdot n)$

#120 Triangle

Description: Find the minimum path sum from top to bottom, moving to adjacent indices on the row below. **Intuition:** working bottom-up, each cell's best is its value plus the cheaper of the two children directly below. **Algorithm:** $dp[j]$ = min path sum from row i cell j to the bottom; collapse one row at a time; answer $dp[0]$.

```

class Solution {
    public int minimumTotal(List<List<Integer>> triangle) {
        int n = triangle.size();
        int[] dp = new int[n + 1];
        for (int i = n - 1; i >= 0; i--) { // ← VARIATION: bottom-up over rows
            for (int j = 0; j <= i; j++) {
                dp[j] = triangle.get(i).get(j) + Math.min(dp[j], dp[j+1]);
            }
        }
        return dp[0];
    }
}

```

Time $O(n^2)$ | **Space** $O(n)$

#132 Palindrome Partitioning II

Description: Return the minimum number of cuts so that every substring of the partition is a palindrome. **Intuition:** if $s[j..i]$ is a palindrome, a cut after $j-1$ lets $cuts[i] = cuts[j-1] + 1$; precompute palindromes with interval DP. **Algorithm:** $pal[i][j]$ palindrome flag; $cuts[i]$ = min cuts for $s[0..i]$; answer $cuts[n-1]$.

```

class Solution {
    public int minCut(String s) {
        int n = s.length();
        boolean[][] pal = new boolean[n][n];
        for (int i = n - 1; i >= 0; i--) { // ← VARIATION: interval DP precompute palindromes
            for (int j = i; j < n; j++) {
                pal[i][j] = s.charAt(i) == s.charAt(j) && (j - i < 2 || pal[i+1][j-1]);
            }
        }
        int[] cuts = new int[n];
        for (int i = 0; i < n; i++) {
            if (pal[0][i]) {
                cuts[i] = 0;
            } else {
                cuts[i] = i;
                for (int j = 1; j <= i; j++) {
                    if (pal[j][i]) {
                        cuts[i] = Math.min(cuts[i], cuts[j-1] + 1); // ← VARIATION: cut before palindrome suffix
                    }
                }
            }
        }
        return cuts[n-1];
    }
}

```

Time $O(n^2)$ | **Space** $O(n^2)$

#140 Word Break II

Description: Return all sentences obtained by segmenting `s` into space-separated dictionary words. **Intuition:** memoize on each suffix the list of valid segmentations, prepending each matching word to the segmentations of the remainder. **Algorithm:** `dfs(start)` returns all sentences for `s[start..]`; cache per `start`; answer `dfs(0)`.

```

class Solution {
    private Map<Integer, List<String>> memo = new HashMap<>();
    private Set<String> words;

    public List<String> wordBreak(String s, List<String> wordDict) {
        words = new HashSet<>(wordDict);
        return dfs(s, 0);
    }

    private List<String> dfs(String s, int start) { // ← VARIATION: memoized DFS returns sentences
        if (memo.containsKey(start)) {
            return memo.get(start);
        }
        List<String> result = new ArrayList<>();
        if (start == s.length()) {
            result.add("");
            return result;
        }
        for (int i = start + 1; i <= s.length(); i++) {
            String word = s.substring(start, i);
            if (words.contains(word)) {
                for (String tail : dfs(s, i)) {
                    result.add(tail.isEmpty() ? word : word + " " + tail);
                }
            }
        }
        memo.put(start, result);
        return result;
    }
}

```

Time $O(n^2 \cdot 2^n)$ worst case | **Space** $O(2^n)$

#152 Maximum Product Subarray

Description: Find the contiguous subarray with the largest product. **Intuition:** a negative number flips max and min, so track both; the running max product ending at `i` may come from the prior max or min. **Algorithm:** maintain `maxEnd` and `minEnd`; swap on negative; track global `result`.

```
class Solution {
    public int maxProduct(int[] nums) {
        int maxEnd = nums[0], minEnd = nums[0], result = nums[0];
        for (int i = 1; i < nums.length; i++) {
            if (nums[i] < 0) { // ← VARIATION: negative swaps max/min
                int temp = maxEnd;
                maxEnd = minEnd;
                minEnd = temp;
            }
            maxEnd = Math.max(nums[i], maxEnd * nums[i]);
            minEnd = Math.min(nums[i], minEnd * nums[i]);
            result = Math.max(result, maxEnd);
        }
        return result;
    }
}
```

Time $O(n)$ | **Space** $O(1)$

#174 Dungeon Game

Description: Find the minimum starting health so the knight survives from top-left to bottom-right (each cell adds/subtracts health, must stay ≥ 1). **Intuition:** health needed depends on the future, so fill from the bottom-right: needed entering a cell = $\max(1, \text{min child need} - \text{cell value})$. **Algorithm:** `dp[i][j]` = min health needed entering `(i,j)`; answer `dp[0][0]`.

```
class Solution {
    public int calculateMinimumHP(int[][] dungeon) {
        int m = dungeon.length, n = dungeon[0].length;
        int[][] dp = new int[m+1][n+1];
        for (int[] row : dp) {
            Arrays.fill(row, Integer.MAX_VALUE);
        }
        dp[m][n-1] = dp[m-1][n] = 1;
        for (int i = m - 1; i >= 0; i--) { // ← VARIATION: fill from bottom-right
            for (int j = n - 1; j >= 0; j--) {
                int need = Math.min(dp[i+1][j], dp[i][j+1]) - dungeon[i][j];
                dp[i][j] = Math.max(1, need); // ← VARIATION: health must stay ≥ 1
            }
        }
        return dp[0][0];
    }
}
```

Time $O(m \cdot n)$ | **Space** $O(m \cdot n)$

#188 Best Time to Buy and Sell Stock IV

Description: Maximize profit with at most `k` transactions. **Intuition:** for each allowed transaction count `t`, track best buy and sell; this is #123 generalized to `k` chained state pairs. **Algorithm:** `buy[t]` / `sell[t]` for `t` in `1..k`; answer `sell[k]`.

```

class Solution {
    public int maxProfit(int k, int[] prices) {
        if (prices.length == 0 || k == 0) {
            return 0;
        }
        int[] buy = new int[k + 1], sell = new int[k + 1];
        Arrays.fill(buy, Integer.MIN_VALUE);
        for (int price : prices) {
            for (int t = 1; t <= k; t++) {
                // ← VARIATION: k chained state pairs
                buy[t] = Math.max(buy[t], sell[t-1] - price);
                sell[t] = Math.max(sell[t], buy[t] + price);
            }
        }
        return sell[k];
    }
}

```

Time $O(n \cdot k)$ | **Space** $O(k)$

#264 Ugly Number II

Description: Find the n th ugly number (positive integers whose only prime factors are 2, 3, 5). **Intuition:** every ugly number is a previous ugly number times 2, 3, or 5; merge the three multiples with pointers. **Algorithm:** $dp[i]$ = i -th ugly number; advance pointers $p2, p3, p5$; answer $dp[n-1]$.

```

class Solution {
    public int nthUglyNumber(int n) {
        int[] dp = new int[n];
        dp[0] = 1;
        int p2 = 0, p3 = 0, p5 = 0;
        for (int i = 1; i < n; i++) {
            // ← VARIATION: merge multiples via pointers
            int next2 = dp[p2] * 2, next3 = dp[p3] * 3, next5 = dp[p5] * 5;
            dp[i] = Math.min(next2, Math.min(next3, next5));
            if (dp[i] == next2) p2++;
            if (dp[i] == next3) p3++;
            if (dp[i] == next5) p5++;
        }
        return dp[n-1];
    }
}

```

Time $O(n)$ | **Space** $O(n)$

#313 Super Ugly Number

Description: Find the n th super ugly number whose prime factors all come from a given `primes` list. **Intuition:** generalize #264 to arbitrary primes — keep one pointer per prime and pick the minimum next candidate. **Algorithm:** $dp[i]$ = i -th super ugly; $pointers[j]$ for each prime; answer $dp[n-1]$.


```

class Solution {
    public int nthSuperUglyNumber(int n, int[] primes) {
        int k = primes.length;
        int[] dp = new int[n], pointers = new int[k];
        dp[0] = 1;
        for (int i = 1; i < n; i++) {
            int min = Integer.MAX_VALUE;
            for (int j = 0; j < k; j++) {
                // ← VARIATION: one pointer per prime
                min = Math.min(min, dp[pointers[j]] * primes[j]);
            }
            dp[i] = min;
            for (int j = 0; j < k; j++) {
                if (dp[pointers[j]] * primes[j] == min) {
                    pointers[j]++;
                }
            }
        }
        return dp[n-1];
    }
}

```

Time $O(n \cdot k)$ | **Space** $O(n+k)$

#337 House Robber III

Description: Houses form a binary tree; you cannot rob two directly-linked houses. Maximize the total. **Intuition:** each node returns two values — best if it is robbed (skip children) vs not robbed (children free to choose). **Algorithm:** `dfs(node)` returns `{notRob, rob}`; answer = `max` of root's pair.

```

class Solution {
    public int rob(TreeNode root) {
        int[] result = dfs(root);
        return Math.max(result[0], result[1]);
    }
    private int[] dfs(TreeNode node) {
        // ← VARIATION: tree DP returns {notRob, rob}
        if (node == null) {
            return new int[]{0, 0};
        }
        int[] left = dfs(node.left), right = dfs(node.right);
        int notRob = Math.max(left[0], left[1]) + Math.max(right[0], right[1]);
        int rob = node.val + left[0] + right[0];
        return new int[]{notRob, rob};
    }
}

```

Time $O(n)$ | **Space** $O(h)$

#343 Integer Break

Description: Break `n` into the sum of at least two positive integers maximizing their product. **Intuition:** for each split `j`, the rest can stay as `i-j` or be broken further (`dp[i-j]`); take the best. **Algorithm:** `dp[i]` = max product of breaking `i` (into ≥ 2 parts); answer `dp[n]`.

```

class Solution {
    public int integerBreak(int n) {
        int[] dp = new int[n + 1];
        dp[1] = 1;
        for (int i = 2; i <= n; i++) {
            for (int j = 1; j < i; j++) {
                // ← VARIATION: split point, break further or not
                dp[i] = Math.max(dp[i], Math.max(j * (i - j), j * dp[i - j]));
            }
        }
        return dp[n];
    }
}

```

Time $O(n^2)$ | **Space** $O(n)$

#375 Guess Number Higher or Lower II

Description: Find the minimum amount of money to guarantee a win when guessing a number in $[1, n]$, paying the guessed value on each wrong guess. **Intuition:** for range $[i, j]$, guessing k costs k plus the worst of the two resulting subranges; minimize over k . **Algorithm:** $dp[i][j]$ = min worst-case cost for $[i, j]$ (interval DP); answer $dp[1][n]$.

```

class Solution {
    public int getMoneyAmount(int n) {
        int[][] dp = new int[n+2][n+2];
        for (int len = 2; len <= n; len++) {
            // ← VARIATION: interval DP over range length
            for (int i = 1; i + len - 1 <= n; i++) {
                int j = i + len - 1;
                dp[i][j] = Integer.MAX_VALUE;
                for (int k = i; k <= j; k++) {
                    // ← VARIATION: guess k, take worst subrange
                    int cost = k + Math.max(dp[i][k-1], dp[k+1][j]);
                    dp[i][j] = Math.min(dp[i][j], cost);
                }
            }
        }
        return dp[1][n];
    }
}

```

Time $O(n^3)$ | **Space** $O(n^2)$

#376 Wiggle Subsequence

Description: Find the length of the longest subsequence whose consecutive differences strictly alternate between positive and negative. **Intuition:** track the best subsequence ending with an up-move and with a down-move; each rise extends down, each fall extends up. **Algorithm:** up / down running lengths; answer $\max(\text{up}, \text{down})$.

```

class Solution {
    public int wiggleMaxLength(int[] nums) {
        int up = 1, down = 1;
        for (int i = 1; i < nums.length; i++) {
            if (nums[i] > nums[i-1]) {
                // ← VARIATION: rise extends down-ending seq
                up = down + 1;
            } else if (nums[i] < nums[i-1]) {
                down = up + 1;
            }
        }
        return Math.max(up, down);
    }
}

```

Time $O(n)$ | **Space** $O(1)$

#413 Arithmetic Slices

Description: Count the number of arithmetic subarrays (length ≥ 3 , constant difference). **Intuition:** if `nums[i]` continues the arithmetic run ending at `i-1`, it adds `dp[i-1]+1` new slices ending at `i`. **Algorithm:** `current` = arithmetic slices ending at `i`; accumulate into `result`.

```
class Solution {
    public int numberOfArithmeticSlices(int[] nums) {
        int current = 0, result = 0;
        for (int i = 2; i < nums.length; i++) {
            if (nums[i] - nums[i-1] == nums[i-1] - nums[i-2]) { // ← VARIATION: extend arithmetic run
                current++;
                result += current;
            } else {
                current = 0;
            }
        }
        return result;
    }
}
```

Time $O(n)$ | **Space** $O(1)$

#446 Arithmetic Slices II - Subsequence

Description: Count the arithmetic subsequences (length ≥ 3) of the array. **Intuition:** for each pair `(j, i)` with difference `d`, weak slices ending at `i` extend those ending at `j`; promote weak to valid when length reaches 3. **Algorithm:** `dp[i]` is a map from difference `d` to count of weak arithmetic subsequences ending at `i`; accumulate `result`.

```
class Solution {
    public int numberOfArithmeticSlices(int[] nums) {
        int n = nums.length, result = 0;
        Map<Long, Integer>[] dp = new HashMap[n]; // ← VARIATION: per-index map keyed by difference
        for (int i = 0; i < n; i++) {
            dp[i] = new HashMap<>();
            for (int j = 0; j < i; j++) {
                long d = (long) nums[i] - nums[j];
                int countJ = dp[j].getOrDefault(d, 0);
                result += countJ; // ← VARIATION: weak at j becomes valid at i
                dp[i].merge(d, countJ + 1, Integer::sum);
            }
        }
        return result;
    }
}
```

Time $O(n^2)$ | **Space** $O(n^2)$

#486 Predict the Winner

Description: Two players alternately take a number from either end; decide whether player 1 can win (score $\geq$ player 2). **Intuition:** on range `[i, j]` the current player maximizes value – opponent's best on the remaining range. **Algorithm:** `dp[i][j]` = best score difference (current minus opponent) for `[i, j]`; answer `dp[0][n-1] >= 0`.

```

class Solution {
    public boolean predictTheWinner(int[] nums) {
        int n = nums.length;
        int[][] dp = new int[n][n];
        for (int i = 0; i < n; i++) {
            dp[i][i] = nums[i];
        }
        for (int i = n - 1; i >= 0; i--) {
            // ← VARIATION: interval DP, score difference
            for (int j = i + 1; j < n; j++) {
                dp[i][j] = Math.max(nums[i] - dp[i+1][j], nums[j] - dp[i][j-1]);
            }
        }
        return dp[0][n-1] >= 0;
    }
}

```

Time $O(n^2)$ | **Space** $O(n^2)$

#509 Fibonacci Number

Description: Return the n th Fibonacci number. **Intuition:** each term is the sum of the previous two — the canonical linear 1D recurrence. **Algorithm:** roll two variables `previous` and `current`; answer after `n` steps.

```

class Solution {
    public int fib(int n) {
        if (n < 2) {
            return n;
        }
        int previous = 0, current = 1;
        for (int i = 2; i <= n; i++) {
            int next = previous + current;
            previous = current;
            current = next;
        }
        return current;
    }
}

```

Time $O(n)$ | **Space** $O(1)$

#552 Student Attendance Record II

Description: Count length-`n` attendance records that are rewardable (at most one 'A', no three consecutive 'L'), modulo $1e9+7$. **Intuition:** state = (number of 'A's so far 0/1, trailing 'L' run 0/1/2); each day append P, A, or L respecting limits. **Algorithm:** `dp[a][l]` counts records ending in that state; roll day by day; answer = sum over all states.

```

class Solution {
    public int checkRecord(int n) {
        int MOD = 1_000_000_007;
        long[][] dp = new long[2][3]; // ← VARIATION: state = (A count, trailing L run)
        dp[0][0] = 1;
        for (int day = 0; day < n; day++) {
            long[][] next = new long[2][3];
            for (int a = 0; a < 2; a++) {
                for (int l = 0; l < 3; l++) {
                    long ways = dp[a][l];
                    if (ways == 0) {
                        continue;
                    }
                    next[a][0] = (next[a][0] + ways) % MOD; // append 'P' resets L run
                    if (a == 0) {
                        next[1][0] = (next[1][0] + ways) % MOD; // append 'A' (only if none yet)
                    }
                    if (l < 2) {
                        next[a][l+1] = (next[a][l+1] + ways) % MOD; // append 'L' (run < 3)
                    }
                }
            }
            dp = next;
        }
        long result = 0;
        for (int a = 0; a < 2; a++) {
            for (int l = 0; l < 3; l++) {
                result = (result + dp[a][l]) % MOD;
            }
        }
        return (int) result;
    }
}

```

Time $O(n)$ | **Space** $O(1)$

#576 Out of Boundary Paths

Description: Count paths that move a ball off the grid in exactly `maxMove` steps from a start cell, modulo $1e9+7$. **Intuition:** moving off the boundary counts as one path; otherwise spread current-step counts to 4 neighbors for the next step. **Algorithm:** `dp[i][j]` = ways to be at `(i,j)` after `step` moves; sum boundary exits; answer = accumulated `result`.

```

class Solution {
    int[] dr = {1,-1,0,0}, dc = {0,0,1,-1};

    public int findPaths(int m, int n, int maxMove, int startRow, int startColumn) {
        int MOD = 1_000_000_007;
        long[][] dp = new long[m][n];
        dp[startRow][startColumn] = 1;
        long result = 0;
        for (int step = 0; step < maxMove; step++) {
            long[][] next = new long[m][n];
            for (int i = 0; i < m; i++) {
                for (int j = 0; j < n; j++) {
                    if (dp[i][j] == 0) {
                        continue;
                    }
                    for (int dir = 0; dir < 4; dir++) { // ← VARIATION: spread to 4 neighbors
                        int ni = i + dr[dir], nj = j + dc[dir];
                        if (ni < 0 || ni >= m || nj < 0 || nj >= n) {
                            result = (result + dp[i][j]) % MOD; // ← VARIATION: stepping off grid = a path
                        } else {
                            next[ni][nj] = (next[ni][nj] + dp[i][j]) % MOD;
                        }
                    }
                }
            }
            dp = next;
        }
        return (int) result;
    }
}

```

Time $O(\text{maxMove} \cdot m \cdot n)$ | **Space** $O(m \cdot n)$

#600 Non-negative Integers without Consecutive Ones

Description: Count integers in $[0, n]$ whose binary representation has no two adjacent 1 bits. **Intuition:** scan bits high-to-low; precompute Fibonacci-style counts of valid suffixes, adding them when a free bit is 1, and stop early if two consecutive 1s appear in n . **Algorithm:** $\text{fib}[i]$ = valid numbers with i free bits; walk bits of n ; answer = accumulated result (+1 for n itself if valid).

```

class Solution {
    public int findIntegers(int n) {
        int[] fib = new int[32]; // ← VARIATION: fib[i] = valid i-bit strings
        fib[0] = 1; fib[1] = 2;
        for (int i = 2; i < 32; i++) {
            fib[i] = fib[i-1] + fib[i-2];
        }
        int result = 0, previousBit = 0;
        for (int i = 30; i >= 0; i--) {
            if ((n & (1 << i)) != 0) { // current bit is 1
                result += fib[i]; // ← VARIATION: count numbers with 0 here
                if (previousBit == 1) { // ← VARIATION: two adjacent 1s in n, stop
                    return result;
                }
                previousBit = 1;
            } else {
                previousBit = 0;
            }
        }
        return result + 1; // include n itself
    }
}

```

Time $O(1)$ (32 bits) | **Space** $O(1)$

#629 K Inverse Pairs Array

Description: Count permutations of $1..n$ with exactly k inverse pairs, modulo $1e9+7$. **Intuition:** inserting n into a permutation of $1..n-1$ adds $0..n-1$ inversions; this gives a sliding-window sum over the previous row. **Algorithm:** $dp[i][j]$ = permutations of i numbers with j inversions, using prefix sums; answer $dp[n][k]$.

```
class Solution {
    public int kInversePairs(int n, int k) {
        int MOD = 1_000_000_007;
        long[][] dp = new long[n+1][k+1];
        dp[0][0] = 1;
        for (int i = 1; i <= n; i++) {
            long[] prefix = new long[k+2];
            for (int j = 0; j <= k; j++) {
                prefix[j+1] = (prefix[j] + dp[i-1][j]) % MOD; // ← VARIATION: prefix sums of previous row
            }
            for (int j = 0; j <= k; j++) { // window [j-(i-1), j] of previous row
                int lo = Math.max(0, j - (i - 1));
                dp[i][j] = (prefix[j+1] - prefix[lo] + MOD) % MOD;
            }
        }
        return (int) dp[n][k];
    }
}
```

Time $O(n \cdot k)$ | **Space** $O(n \cdot k)$

#638 Shopping Offers

Description: Buy exactly $needs[i]$ of each item at minimum cost, given unit prices and special bundle offers (each usable multiple times). **Intuition:** treat the remaining needs vector as the state; either buy everything at unit price, or apply an affordable offer and recurse. **Algorithm:** memoized DFS over the needs list; answer $dfs(needs)$.

```

class Solution {
    private Map<List<Integer>, Integer> memo = new HashMap<>();

    public int shoppingOffers(List<Integer> price, List<List<Integer>> special, List<Integer> needs) {
        return dfs(price, special, needs);
    }

    private int dfs(List<Integer> price, List<List<Integer>> special, List<Integer> needs) { // ← VARIATION: DFS+memo over needs ve
        if (memo.containsKey(needs)) {
            return memo.get(needs);
        }
        int n = price.size(), best = 0;
        for (int i = 0; i < n; i++) {
            best += needs.get(i) * price.get(i); // baseline: all at unit price
        }
        for (List<Integer> offer : special) {
            List<Integer> remaining = new ArrayList<>();
            boolean valid = true;
            for (int i = 0; i < n; i++) {
                int left = needs.get(i) - offer.get(i);
                if (left < 0) { // ← VARIATION: offer must not overbuy
                    valid = false;
                    break;
                }
                remaining.add(left);
            }
            if (valid) {
                best = Math.min(best, offer.get(n) + dfs(price, special, remaining));
            }
        }
        memo.put(needs, best);
        return best;
    }
}

```

Time $O(\text{offers} \cdot \prod \text{needs}_i)$ | **Space** $O(\prod \text{needs}_i)$

#639 Decode Ways II

Description: Count decodings of a digit string where `*` is a wildcard for digits 1-9, modulo $1e9+7$. **Intuition:** extend #91 — each position can decode one char (count 1-9 options) or two chars (count valid 10-26 combinations); `*` multiplies the options. **Algorithm:** $dp[i]$ = decodings of $s[0..i]$; answer $dp[n]$.


```

class Solution {
    public int numDecodings(String s) {
        int MOD = 1_000_000_007, n = s.length();
        long[] dp = new long[n + 1];
        dp[0] = 1;
        dp[1] = oneDigit(s.charAt(0));
        for (int i = 2; i <= n; i++) {
            char current = s.charAt(i-1), prev = s.charAt(i-2);
            dp[i] = (dp[i] + dp[i-1] * oneDigit(current)) % MOD; // ← VARIATION: single char, count options
            dp[i] = (dp[i] + dp[i-2] * twoDigit(prev, current)) % MOD; // ← VARIATION: pair, count 10-26 options
        }
        return (int) dp[n];
    }
    private long oneDigit(char c) {
        if (c == '*') return 9; // 1-9
        return c == '0' ? 0 : 1;
    }
    private long twoDigit(char a, char b) {
        if (a == '*' && b == '*') return 15; // 11-19, 21-26
        if (a == '*') { // *X
            return b <= '6' ? 2 : 1; // 1X always, 2X only if X<=6
        }
        if (b == '*') { // X*
            if (a == '1') return 9; // 11-19
            if (a == '2') return 6; // 21-26
            return 0;
        }
        int value = (a - '0') * 10 + (b - '0');
        return value >= 10 && value <= 26 ? 1 : 0;
    }
}

```

Time $O(n)$ | **Space** $O(n)$

#650 2 Keys Keyboard

Description: Starting with one 'A', using only Copy-All and Paste, return the minimum operations to obtain n 'A's. **Intuition:** the answer is the sum of n 's prime factors — building i from a divisor j costs i/j operations. **Algorithm:** $dp[i]$ = min operations to reach i characters; for each divisor j , $dp[i] = dp[j] + i/j$; answer $dp[n]$.

```

class Solution {
    public int minSteps(int n) {
        int[] dp = new int[n + 1];
        for (int i = 2; i <= n; i++) {
            dp[i] = i; // worst case: copy then paste i-1 times
            for (int j = i / 2; j >= 1; j--) { // ← VARIATION: build i from divisor j
                if (i % j == 0) {
                    dp[i] = dp[j] + i / j;
                    break; // largest divisor gives the min
                }
            }
        }
        return dp[n];
    }
}

```

Time $O(n\sqrt{n})$ | **Space** $O(n)$

#673 Number of Longest Increasing Subsequence

Description: Count the number of longest strictly increasing subsequences. **Intuition:** alongside LIS length, track how many ways achieve it; a longer extension resets the count, an equal-length extension accumulates it. **Algorithm:** $length[i]$ and $count[i]$ for LIS ending at i ; sum counts of maximum-length endings; answer $result$.

```

class Solution {
    public int findNumberOfLIS(int[] nums) {
        int n = nums.length, maxLen = 0, result = 0;
        int[] length = new int[n], count = new int[n];
        for (int i = 0; i < n; i++) {
            length[i] = 1; count[i] = 1;
            for (int j = 0; j < i; j++) {
                if (nums[j] < nums[i]) {
                    if (length[j] + 1 > length[i]) {           // ← VARIATION: longer extension resets count
                        length[i] = length[j] + 1;
                        count[i] = count[j];
                    } else if (length[j] + 1 == length[i]) {    // ← VARIATION: equal length accumulates count
                        count[i] += count[j];
                    }
                }
            }
            if (length[i] > maxLen) {
                maxLen = length[i];
                result = count[i];
            } else if (length[i] == maxLen) {
                result += count[i];
            }
        }
        return result;
    }
}

```

Time $O(n^2)$ | **Space** $O(n)$

#674 Longest Continuous Increasing Subsequence

Description: Find the length of the longest continuous (contiguous) strictly increasing subarray. **Intuition:** extend the current run while values rise, otherwise restart; a simpler linear scan of #300. **Algorithm:** `current` run length; track `result` .

```

class Solution {
    public int findLengthOfLCIS(int[] nums) {
        int current = 1, result = 1;
        for (int i = 1; i < nums.length; i++) {
            if (nums[i] > nums[i-1]) {
                current++;           // ← VARIATION: contiguous run, reset on drop
            } else {
                current = 1;
            }
            result = Math.max(result, current);
        }
        return result;
    }
}

```

Time $O(n)$ | **Space** $O(1)$

#688 Knight Probability in Chessboard

Description: Return the probability that a knight remains on an $n \times n$ board after exactly `k` moves from a start cell, each move chosen uniformly among 8. **Intuition:** spread the probability at each cell to its 8 knight-move targets per step; off-board moves lose probability. **Algorithm:** `dp[i][j]` = probability of being at `(i,j)` after `step` moves; answer = sum of final grid.

```

class Solution {
    int[] dr = {2,2,-2,-2,1,1,-1,-1}, dc = {1,-1,1,-1,2,-2,2,-2};

    public double knightProbability(int n, int k, int row, int column) {
        double[][] dp = new double[n][n];
        dp[row][column] = 1.0;
        for (int step = 0; step < k; step++) {
            double[][] next = new double[n][n];
            for (int i = 0; i < n; i++) {
                for (int j = 0; j < n; j++) {
                    if (dp[i][j] == 0) {
                        continue;
                    }
                    for (int dir = 0; dir < 8; dir++) { // ← VARIATION: 8 knight moves, each prob 1/8
                        int ni = i + dr[dir], nj = j + dc[dir];
                        if (ni >= 0 && ni < n && nj >= 0 && nj < n) {
                            next[ni][nj] += dp[i][j] / 8.0;
                        }
                    }
                }
            }
            dp = next;
        }
        double result = 0;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                result += dp[i][j];
            }
        }
        return result;
    }
}

```

Time $O(k \cdot n^2)$ | **Space** $O(n^2)$

#718 Maximum Length of Repeated Subarray

Description: Find the length of the longest subarray common to two arrays (contiguous). **Intuition:** unlike LCS, the match must be contiguous, so a mismatch resets the running length to 0. **Algorithm:** $dp[i][j]$ = length of common suffix ending at $nums1[i-1]$, $nums2[j-1]$; track global result.

```

class Solution {
    public int findLength(int[] nums1, int[] nums2) {
        int m = nums1.length, n = nums2.length, result = 0;
        int[][] dp = new int[m+1][n+1];
        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (nums1[i-1] == nums2[j-1]) {
                    dp[i][j] = dp[i-1][j-1] + 1; // ← VARIATION: contiguous, no carry on mismatch
                    result = Math.max(result, dp[i][j]);
                }
            }
        }
        return result;
    }
}

```

Time $O(m \cdot n)$ | **Space** $O(m \cdot n)$

#740 Delete and Earn

Description: Deleting $nums[i]$ earns its value but removes all copies of $nums[i]-1$ and $nums[i]+1$; maximize earnings. **Intuition:** bucket points by value, then it becomes House Robber over the value line — you cannot take adjacent values. **Algorithm:** $points[v]$ = total points for value v ;

`dp[v] = max earnings up to value v ; answer dp[max] .`

```
class Solution {
    public int deleteAndEarn(int[] nums) {
        int max = 0;
        for (int num : nums) {
            max = Math.max(max, num);
        }
        int[] points = new int[max + 1];           // ← VARIATION: bucket points by value
        for (int num : nums) {
            points[num] += num;
        }
        int[] dp = new int[max + 1];
        dp[0] = 0;
        dp[1] = points[1];
        for (int v = 2; v <= max; v++) {           // ← VARIATION: House Robber over value line
            dp[v] = Math.max(dp[v-1], dp[v-2] + points[v]);
        }
        return dp[max];
    }
}
```

Time $O(n + \max)$ | **Space** $O(\max)$

#873 Length of Longest Fibonacci Subsequence

Description: Given a strictly increasing array, find the length of the longest Fibonacci-like subsequence (each term = sum of the previous two).

Intuition: a fib-like subseq is determined by its last two elements (j, i) ; look up whether the required predecessor `arr[i]-arr[j]` exists earlier.

Algorithm: `dp[j][i]` = length of fib seq ending with `arr[j], arr[i]` ; answer = max found (≥ 3).

```
class Solution {
    public int lenLongestFibSubseq(int[] arr) {
        int n = arr.length, result = 0;
        Map<Integer, Integer> index = new HashMap<>();
        for (int i = 0; i < n; i++) {
            index.put(arr[i], i);
        }
        int[][] dp = new int[n][n];
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < i; j++) {
                int needed = arr[i] - arr[j];           // ← VARIATION: predecessor before arr[j]
                Integer k = index.get(needed);
                if (k != null && k < j) {
                    dp[j][i] = dp[k][j] + 1;
                } else {
                    dp[j][i] = 2;                       // seed pair
                }
                if (dp[j][i] > 2) {
                    result = Math.max(result, dp[j][i]);
                }
            }
        }
        return result;
    }
}
```

Time $O(n^2)$ | **Space** $O(n^2)$

#887 Super Egg Drop

Description: With k eggs and n floors, return the minimum number of moves to determine the critical floor in the worst case. **Intuition:** invert the problem — `dp[m][k]` = highest floor count solvable with m moves and k eggs; a move either breaks (test below) or survives (test above), plus the current floor. **Algorithm:** grow m until `dp[m][k] >= n` ; answer = that m .

```

class Solution {
    public int superEggDrop(int k, int n) {
        int[][] dp = new int[n+1][k+1]; // ← VARIATION: dp[m][k] = floors solvable in m moves
        int m = 0;
        while (dp[m][k] < n) {
            m++;
            for (int eggs = 1; eggs <= k; eggs++) {
                dp[m][eggs] = dp[m-1][eggs-1] + dp[m-1][eggs] + 1; // break + survive + current floor
            }
        }
        return m;
    }
}

```

Time $O(k \log n)$ | **Space** $O(n \cdot k)$

#918 Maximum Sum Circular Subarray

Description: Find the maximum subarray sum where the array is circular (subarray may wrap around). **Intuition:** the answer is either a normal max subarray (Kadane) or the total minus the minimum subarray (wrap); guard the all-negative case. **Algorithm:** track max and min running subarrays; answer = $\max(\text{maxSum}, \text{total} - \text{minSum})$ unless all negative.

```

class Solution {
    public int maxSubarraySumCircular(int[] nums) {
        int total = 0, maxSum = nums[0], curMax = 0, minSum = nums[0], curMin = 0;
        for (int num : nums) {
            curMax = Math.max(curMax + num, num);
            maxSum = Math.max(maxSum, curMax);
            curMin = Math.min(curMin + num, num); // ← VARIATION: track min subarray for wrap
            minSum = Math.min(minSum, curMin);
            total += num;
        }
        if (maxSum < 0) { // ← VARIATION: all negative, wrap invalid
            return maxSum;
        }
        return Math.max(maxSum, total - minSum);
    }
}

```

Time $O(n)$ | **Space** $O(1)$

#935 Knight Dialer

Description: Count distinct phone numbers of length n dialable by knight moves on a phone keypad, modulo $1e9+7$. **Intuition:** from each digit a knight can reach a fixed set of digits; spread counts across the adjacency map each step. **Algorithm:** $\text{dp}[d]$ = count of numbers of current length ending at digit d ; answer = sum after n steps.

```

class Solution {
    public int knightDialer(int n) {
        int MOD = 1_000_000_007;
        int[][] moves = {
            {4,6},{6,8},{7,9},{4,8},{0,3,9},{},
            {0,1,7},{2,6},{1,3},{2,4}
        };
        long[] dp = new long[10];
        Arrays.fill(dp, 1);
        for (int step = 1; step < n; step++) {
            long[] next = new long[10];
            for (int d = 0; d < 10; d++) {
                for (int target : moves[d]) {
                    next[target] = (next[target] + dp[d]) % MOD;
                }
            }
            dp = next;
        }
        long result = 0;
        for (long count : dp) {
            result = (result + count) % MOD;
        }
        return (int) result;
    }
}

```

Time $O(n)$ | **Space** $O(1)$

#983 Minimum Cost For Tickets

Description: Given travel days and the cost of 1-day, 7-day, and 30-day passes, return the minimum cost to cover all travel days. **Intuition:** on a travel day choose the cheapest pass covering it by looking back 1, 7, or 30 days; non-travel days inherit the prior cost. **Algorithm:** $dp[i]$ = min cost through day i ; answer $dp[\text{lastDay}]$.

```

class Solution {
    public int mincostTickets(int[] days, int[] costs) {
        int last = days[days.length - 1];
        boolean[] travel = new boolean[last + 1];
        for (int day : days) {
            travel[day] = true;
        }
        int[] dp = new int[last + 1];
        for (int i = 1; i <= last; i++) {
            if (!travel[i]) {
                dp[i] = dp[i-1];
            } else {
                dp[i] = Math.min(dp[i-1] + costs[0],
                    Math.min(dp[Math.max(0, i-7)] + costs[1],
                        dp[Math.max(0, i-30)] + costs[2]));
            }
        }
        return dp[last];
    }
}

```

Time $O(\text{lastDay})$ | **Space** $O(\text{lastDay})$

#1027 Longest Arithmetic Subsequence

Description: Find the length of the longest arithmetic subsequence (constant difference). **Intuition:** for each pair (j, i) , the arithmetic subseq ending at i with difference d extends the one ending at j . **Algorithm:** $dp[i]$ maps difference d to subseq length ending at i ; track global result.

```

class Solution {
    public int longestArithSeqLength(int[] nums) {
        int n = nums.length, result = 0;
        Map<Integer, Integer>[] dp = new HashMap[n]; // ← VARIATION: per-index map keyed by difference
        for (int i = 0; i < n; i++) {
            dp[i] = new HashMap<>();
            for (int j = 0; j < i; j++) {
                int d = nums[i] - nums[j];
                int length = dp[j].getOrDefault(d, 1) + 1;
                dp[i].put(d, length);
                result = Math.max(result, length);
            }
        }
        return result;
    }
}

```

Time $O(n^2)$ | **Space** $O(n^2)$

#1048 Longest String Chain

Description: Find the longest chain of words where each word is a predecessor of the next (insert exactly one char). **Intuition:** sort by length; for each word, try removing each character to find a shorter predecessor and extend its best chain. **Algorithm:** `dp` maps word to longest chain ending at it; answer = max chain length.

```

class Solution {
    public int longestStrChain(String[] words) {
        Arrays.sort(words, (a, b) -> a.length() - b.length()); // ← VARIATION: process shortest first
        Map<String, Integer> dp = new HashMap<>();
        int result = 0;
        for (String word : words) {
            int best = 1;
            for (int i = 0; i < word.length(); i++) { // ← VARIATION: drop one char to find predecessor
                StringBuilder builder = new StringBuilder(word);
                builder.deleteCharAt(i);
                String predecessor = builder.toString();
                if (dp.containsKey(predecessor)) {
                    best = Math.max(best, dp.get(predecessor) + 1);
                }
            }
            dp.put(word, best);
            result = Math.max(result, best);
        }
        return result;
    }
}

```

Time $O(n \cdot L^2)$ | **Space** $O(n)$

#1137 N-th Tribonacci Number

Description: Return the n th Tribonacci number (each term is the sum of the previous three). **Intuition:** the linear 1D recurrence with three rolling variables instead of two. **Algorithm:** roll `a`, `b`, `c`; answer after `n` steps.

```

class Solution {
    public int tribonacci(int n) {
        if (n == 0) {
            return 0;
        }
        if (n <= 2) {
            return 1;
        }
        int a = 0, b = 1, c = 1;
        for (int i = 3; i <= n; i++) {
            int next = a + b + c;           // ← VARIATION: sum of previous three
            a = b;
            b = c;
            c = next;
        }
        return c;
    }
}

```

Time $O(n)$ | **Space** $O(1)$

#1216 Valid Palindrome III

Description: Return whether `s` becomes a palindrome after removing at most `k` characters. **Intuition:** the minimum removals to make `s` a palindrome equals `n - LPS(s)`; check whether that is $\leq k$. **Algorithm:** interval DP for longest palindromic subsequence; answer `n - dp[0][n-1] <= k`.

```

class Solution {
    public boolean isValidPalindrome(String s, int k) {
        int n = s.length();
        int[][] dp = new int[n][n];           // dp[i][j] = LPS in s[i..j]
        for (int i = n - 1; i >= 0; i--) {    // ← VARIATION: interval DP for LPS
            dp[i][i] = 1;
            for (int j = i + 1; j < n; j++) {
                if (s.charAt(i) == s.charAt(j)) {
                    dp[i][j] = dp[i+1][j-1] + 2;
                } else {
                    dp[i][j] = Math.max(dp[i+1][j], dp[i][j-1]);
                }
            }
        }
        return n - dp[0][n-1] <= k;          // ← VARIATION: removals = n - LPS
    }
}

```

Time $O(n^2)$ | **Space** $O(n^2)$

#1218 Longest Arithmetic Subsequence of Given Difference

Description: Find the longest arithmetic subsequence with a fixed difference `difference`. **Intuition:** with a known difference, the predecessor of `num` must be `num - difference`; one hashmap pass suffices. **Algorithm:** `dp` maps value to longest chain ending at it; answer = max length.


```

class Solution {
    public int longestSubsequence(int[] arr, int difference) {
        Map<Integer, Integer> dp = new HashMap<>(); // ← VARIATION: fixed difference, value-keyed map
        int result = 0;
        for (int num : arr) {
            int length = dp.getOrDefault(num - difference, 0) + 1;
            dp.put(num, length);
            result = Math.max(result, length);
        }
        return result;
    }
}

```

Time $O(n)$ | **Space** $O(n)$

#1269 Number of Ways to Stay in the Same Place After Some Steps

Description: Count the ways to return to index 0 after `steps` moves (stay, left, or right) without leaving an array of size `arrLen`, modulo $1e9+7$.

Intuition: position can never exceed $\text{steps}/2$, so cap the reachable range; each step a position spreads to stay/left/right. **Algorithm:** `dp[i]` = ways to be at index `i` after `step` moves; answer `dp[0]`.

```

class Solution {
    public int numWays(int steps, int arrLen) {
        int MOD = 1_000_000_007;
        int maxPos = Math.min(arrLen - 1, steps / 2); // ← VARIATION: cap reachable positions
        long[] dp = new long[maxPos + 1];
        dp[0] = 1;
        for (int step = 0; step < steps; step++) {
            long[] next = new long[maxPos + 1];
            for (int i = 0; i <= maxPos; i++) {
                if (dp[i] == 0) {
                    continue;
                }
                next[i] = (next[i] + dp[i]) % MOD; // stay
                if (i > 0) {
                    next[i-1] = (next[i-1] + dp[i]) % MOD; // left
                }
                if (i < maxPos) {
                    next[i+1] = (next[i+1] + dp[i]) % MOD; // right
                }
            }
            dp = next;
        }
        return (int) dp[0];
    }
}

```

Time $O(\text{steps} - \min(\text{arrLen}, \text{steps}))$ | **Space** $O(\min(\text{arrLen}, \text{steps}))$

#1395 Count Number of Teams

Description: Count triplets (i, j, k) with $i < j < k$ whose ratings are strictly increasing or strictly decreasing. **Intuition:** fix the middle soldier `j`; the answer is its number of smaller-before $\times$ larger-after, plus the mirror case. **Algorithm:** for each `j` count smaller/larger on both sides; accumulate `result`.

```

class Solution {
    public int numTeams(int[] rating) {
        int n = rating.length, result = 0;
        for (int j = 0; j < n; j++) { // ← VARIATION: fix middle soldier
            int lessLeft = 0, greaterLeft = 0, lessRight = 0, greaterRight = 0;
            for (int i = 0; i < j; i++) {
                if (rating[i] < rating[j]) lessLeft++;
                else if (rating[i] > rating[j]) greaterLeft++;
            }
            for (int k = j + 1; k < n; k++) {
                if (rating[k] < rating[j]) lessRight++;
                else if (rating[k] > rating[j]) greaterRight++;
            }
            result += lessLeft * greaterRight + greaterLeft * lessRight; // ← VARIATION: increasing + decreasing
        }
        return result;
    }
}

```

Time $O(n^2)$ | **Space** $O(1)$

#1567 Maximum Length of Subarray With Positive Product

Description: Find the length of the longest subarray whose product is positive. **Intuition:** track the lengths of positive-product and negative-product runs ending at i ; a negative number swaps them, a zero resets both. **Algorithm:** `positive` / `negative` running lengths; track `result`.

```

class Solution {
    public int getMaxLen(int[] nums) {
        int positive = 0, negative = 0, result = 0;
        for (int num : nums) {
            if (num == 0) { // ← VARIATION: zero resets both runs
                positive = 0;
                negative = 0;
            } else if (num > 0) {
                positive++;
                negative = negative > 0 ? negative + 1 : 0;
            } else { // ← VARIATION: negative swaps pos/neg lengths
                int newPositive = negative > 0 ? negative + 1 : 0;
                int newNegative = positive + 1;
                positive = newPositive;
                negative = newNegative;
            }
            result = Math.max(result, positive);
        }
        return result;
    }
}

```

Time $O(n)$ | **Space** $O(1)$

#1696 Jump Game VI

Description: From index 0, each move jumps 1 to k indices forward; maximize the sum of values landed on, reaching the last index. **Intuition:** `dp[i]` = best score reaching i = `nums[i] + max dp[j]` in the window `[i-k, i-1]`; a monotonic queue keeps that window max in $O(1)$. **Algorithm:** `dp[i]` over a sliding-window-max queue of indices; answer `dp[n-1]`.

```

class Solution {
    public int maxResult(int[] nums, int k) {
        int n = nums.length;
        int[] dp = new int[n];
        dp[0] = nums[0];
        Deque<Integer> queue = new ArrayDeque<>(); // ← VARIATION: monotonic queue of indices
        queue.offerLast(0);
        for (int i = 1; i < n; i++) {
            while (!queue.isEmpty() && queue.peekFirst() < i - k) {
                queue.pollFirst(); // ← VARIATION: drop indices outside window
            }
            dp[i] = nums[i] + dp[queue.peekFirst()]; // window max is at front
            while (!queue.isEmpty() && dp[queue.peekLast()] <= dp[i]) {
                queue.pollLast(); // maintain decreasing dp values
            }
            queue.offerLast(i);
        }
        return dp[n-1];
    }
}

```

Time $O(n)$ | **Space** $O(n)$

#1884 Egg Drop With 2 Eggs and N Floors

Description: With exactly 2 eggs and n floors, return the minimum number of moves to determine the critical floor in the worst case. **Intuition:** $dp[i]$ = min worst-case moves for i floors; dropping at floor j costs $1 + \max(dp[j-1]$ from break, $dp[i-j]$ from survive) . **Algorithm:** minimize over first-drop floor j ; answer $dp[n]$.

```

class Solution {
    public int twoEggDrop(int n) {
        int[] dp = new int[n + 1];
        for (int i = 1; i <= n; i++) {
            dp[i] = i; // worst case with 1 egg behavior
            for (int j = 1; j <= i; j++) {
                dp[i] = Math.min(dp[i], 1 + Math.max(j - 1, dp[i - j])); // ← VARIATION: first drop at floor j
            }
        }
        return dp[n];
    }
}

```

Time $O(n^2)$ | **Space** $O(n)$